

VYOMANAUT

Demonstration Report

Design and Field Measurements of a Decentralised
Cold-Storage Network on a Ten-Machine Campus Rig

AUTHORS

Aryan Singh · Amaresh Singh
Vyomanaunt Labs

Document	VLD-001
Version	1.0.0
Date	29 September 2026
Status	Released
Classification	Public

This report describes the demonstration build of Vyomanaunt — a network that stores encrypted, erasure-coded fragments of files on the idle disks of ordinary desktop computers — and records, with stated provenance, how that build behaved when run on nine laboratory desktops and one laptop on 22 September 2026.

Document control

Version	Date	Change
0.9.0	23 Sep 2026	Internal preliminary draft. Not released; superseded in full by this version.
1.0.0	29 Sep 2026	First public release. Scope limited to the demonstration build and its measured behaviour; all values re-derived from the run's exported time-series database under the rules of Section 4.7.

Status. Released. Every value in this report is traceable to the run's exported Prometheus time-series database, to an output printed by one of the system's own programs, or to a manual instrument reading recorded during the run. Where a value was not obtained, the report says so and labels the quantity an *unmetered parameter* rather than estimating it ([Notation and conventions](#)).

Supersession. This report replaces the previously planned demonstration summary and supersedes every claim about the demonstration build's behaviour or performance made in the Vyomanaunt_Research and Vyomanaunt_V2 repositories, in their design documents, and in any earlier draft of this report. Where a figure in those sources conflicts with a figure here, this report governs.

Redaction. Screenshots reproduced here show only synthetic demonstration identities (provider numbers of the form +9112345678xx and synthetic network identifiers SIM-AS1...SIM-AS9). Account secrets, recovery phrases, one-time codes, administrative credentials and personal contact details captured during the run are not reproduced.

How to cite. A. Singh and A. Singh, *VLD-001: Vyomanaunt Demonstration Report — Design and Field Measurements of a Decentralised Cold-Storage Network on a Ten-Machine Campus Rig*, Version 1.0.0. Vyomanaunt Labs, September 2026.

Copyright. © 2026 Vyomanaunt Labs. This document may be shared and quoted with attribution.

Abstract

Vyomanaunt is a decentralised cold-storage network designed for India. A data owner's device encrypts each file with an all-or-nothing transform, splits it with a Reed–Solomon code, and sends the resulting fragments directly to the idle disks of ordinary desktop computers. A coordinating service admits and vets those desktops, decides where each fragment may be placed, audits every stored fragment continuously, rebuilds redundancy when a desktop leaves, and settles payment from an escrow ledger — without ever carrying or being able to read file data. This report describes the demonstration build of the system and presents measurements from a field run on 22 September 2026, in which nine Windows desktops in a campus computer laboratory acted as storage providers and one laptop acted as both coordinator and data owner, all joined by a ZeroTier overlay network.

The fleet reached upload readiness 35 minutes after the coordinator started and full activation of all nine providers after 58 minutes. Files of 3 to 100 MiB uploaded at 1.1–2.1 MiB/s, reaching 81 % of the ceiling imposed by the data owner's uplink, and were retrieved at 0.6–1.0 MiB/s; a 200 MiB file of 1,335 fragments was stored in full. After every upload the coordinator's record of assigned fragments and the providers' record of stored fragments converged to exactly zero difference. By the end of the run 73,833 storage audits had been issued; the audit pass rate was 94.0 % while small files were being uploaded, 77.9 % while the fragments of the 200 MiB file were still being delivered, and 98.1 % at rest, and no provider ever reported a corrupted fragment. The owner's escrow ledger was exact to the paisa at every one of seven checkpoints, and the placement rule that limits how much of any segment one network may hold was never violated. When a provider announced its departure, 404 repair jobs were queued automatically and began completing at about sixteen per minute. Quantities that the run did not capture are declared as unmetered parameters rather than estimated.

Keywords decentralised storage · erasure coding · all-or-nothing transform · proof of storage · peer-to-peer networks · volunteer resources · measurement study

Results at a glance

The table below collects the principal quantities measured in the run. Identifiers are those of the metric register (Appendix A); provenance tags ^[A], ^[M] and ^[D] are defined in [Notation and conventions](#).

Table 1. Principal measured quantities, Run 3 (and Run 4 where marked).

ID	Quantity	Measured value
D-NET-01	Coordinator start → network ready (5 vetted providers)	35 min 55 s ^[D]
D-NET-02	Coordinator start → all nine providers active	57 min 35 s ^[D]
D-NET-03	Provider join → active ($n = 9$)	median 13 min 11 s; range 10:03–33:02 ^[D]
D-UPL-02	Upload throughput, 3–100 MiB files	1.1–2.1 MiB/s ^[A]
D-RET-01	Retrieval throughput, 3–100 MiB files	0.6–1.0 MiB/s ^[A]
D-UPL-06	Largest file stored	200 MiB (1,335 fragments), 2 resumptions ^[D]
D-UPL-07	Assigned minus confirmed-stored fragments	returned to 0 after every upload ^[A]
D-AUD-01	Audit challenges issued (to 16:50)	73,833 ^[A]
D-AUD-02	Audit pass rate, steady state	94.0 % (ladder); 98.1 % (at rest) ^[A]
D-AUD-06	Content-hash failures on providers	0 ^[A]
D-PAY-01	Owner ledger drift over 7 checkpoints	0 paise ^[D]
D-PLC-01	Diversity-cap violations in inspected placements	0 of 41 segments ^[M]
D-REP-02	Repair jobs queued on one departure (Run 4)	404 ^[A]
D-RES-05/06	Peak memory: coordinator / provider daemon	43 MB / 143–223 MB ^[A]
D-OBS-02	Metrics scrape success	coordinator 99.85 %; providers 99.28–99.68 % ^[A]

Headline finding

Every mechanism the demonstration was built to exhibit — onboarding and vetting, client-side encryption and erasure coding, placement under a network-diversity cap, continuous auditing, exact escrow accounting and automated repair on departure — operated end to end on ten physically separate machines, with the coordinator never carrying a byte of file data. Every fragment the coordinator assigned was confirmed stored, no stored fragment failed an integrity check, and the ledger never drifted by a single paisa.

Not completed in this run. Byte-exact verification of retrieved files (D-RET-02), retrieval of the 200 MiB file, retrieval with providers absent (D-RET-03) and completion of the repair queue (43 of 404 jobs when the campus network failed; D-REP-03) were not obtained (Appendix A).

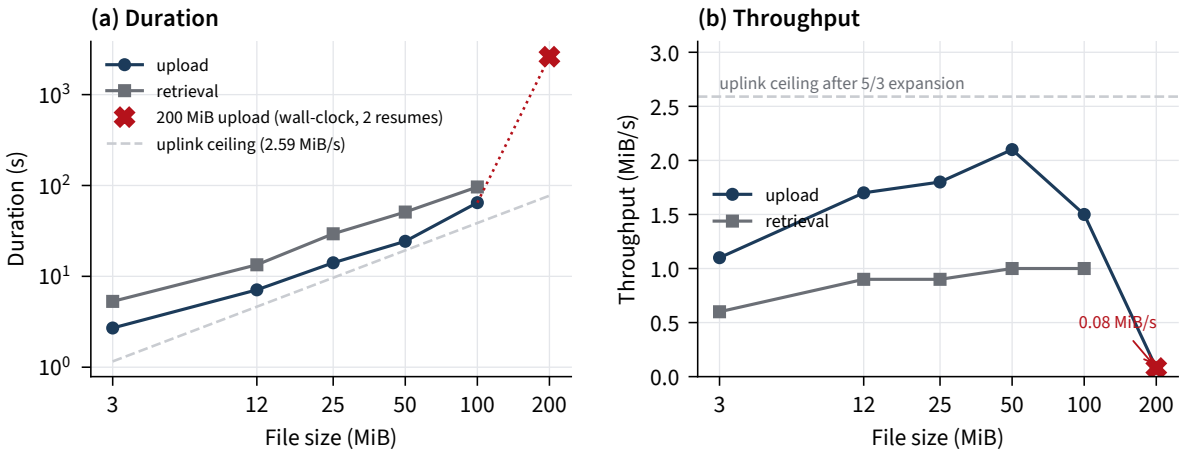


Figure 1. Upload and retrieval across the file-size ladder (Run 3). The dashed grey line is the ceiling imposed by the data owner’s 36.2 Mbps mobile-hotspot uplink after the 5/3 expansion of erasure coding. The 200 MiB point is wall-clock time from start to completion, including two resumptions of the transfer.

Contents

Document control	ii
Abstract	iii
Results at a glance	iv
Notation and conventions	ix
1 Introduction	1
1.1 Motivation	1
1.2 The demonstration build	1
1.3 Contributions of this report	1
1.4 Scope	2
1.5 Organisation	2
2 Background and related work	2
2.1 Decentralised storage networks.	2
2.2 Desktop computers as a storage substrate	2
2.3 Erasure coding and repair	3
2.4 Confidentiality of dispersed data	3
2.5 Proofs of storage	4
2.6 Reputation, incentives and payment	4
2.7 Coordination, discovery and storage engines	4
3 The demonstration build	4
3.1 Roles and the two planes	4
3.2 Encoding pipeline and keys	5
3.3 Transport and discovery	6
3.4 Admission, vetting and the readiness gate	6
3.5 Placement.	6
3.6 Auditing and scoring	7
3.7 Departure and repair.	7
3.8 Escrow and payment.	7
3.9 Provider storage engine	7
3.10 Observability	8
3.11 The demonstration profile.	8
4 Experimental setup and method	8
4.1 Machines	8
4.2 Network	9
4.3 Overlay	9
4.4 Software.	10
4.5 Runs	10
4.6 Workload and procedure	10
4.7 Measurement provenance and aggregation	11
4.8 Measurement conditions	11
5 Results	11
5.1 Network bring-up.	11
5.2 Upload and retrieval across the file-size ladder	13
5.3 Assignment and confirmed delivery	14
5.4 Resource behaviour	16
5.5 Auditing	18
5.6 Repair (Run 4, partial)	19

5.7	Escrow and payment accuracy	22
5.8	Placement.	22
5.9	Churn	23
5.10	Confidentiality and integrity.	23
5.11	Observability	24
6	Future directions and conclusion	24
6.1	Future directions	24
6.2	Conclusion	24
	Acknowledgements	24
A	Metric register	24
B	Raw run data	26
	Glossary	27
	References	30

List of Figures

1	Upload and retrieval across the file-size ladder	v
2	Roles and planes of the demonstration build.	5
3	Encoding pipeline of the demonstration profile	6
4	Per-provider onboarding, join to active	12
5	Operator console at the two bring-up milestones	12
6	Assigned versus confirmed-stored fragments	15
7	Resident memory of the coordinator and of the nine provider daemons (Run 3)..	17
8	Audit results per second by outcome	18
9	Repair queue depth and completed repair jobs after C32 announced its departure.	19
10	Operator console after the announced departure (Run 4)	20
11	File fragments held per provider.	23

List of Tables

1	Principal measured quantities, Run 3 (and Run 4 where marked).	iv
2	Demonstration profile, as printed by the running programs ^[A]	8
3	Machine inventory, recorded at rig start ^[M]	9
4	Network characterisation (TRAI MySpeed), start-of-run pass ^[M]	9
5	Software under test and instrumentation.	10
6	Runs on 22 September 2026	10
7	Network bring-up metrics (Run 3)	13
8	File-size ladder (Run 3)	14
9	System-wide bytes sent per upload (Activity Monitor, 0.01 GB resolution) ^[M]	14
10	Upload and retrieval metrics (Run 3)	16
11	Resource metrics (Run 3)	17
12	Audit outcomes by system state (exact differences of the result counters) ^[A]	18
13	Audit metrics (Run 3)	19
14	Repair and durability metrics (Run 4, partial).	21
15	Owner ledger checkpoints (client balance output) ^[A]	22
16	Payment metrics (Run 3)	22
17	Placement metrics (Run 3)	23
18	Observability metrics (Run 3)	24
19	Metric register and status in Version 1.0.0	25
20	Scribe log, Run 3.	26
21	Assignment counter and confirmed stores at rung boundaries (Prometheus, 15 s samples) ^[A]	26

Notation and conventions

Sizes	The upload client prints sizes and rates in units of 2^{20} bytes; this report writes MiB for those values. The test files are exactly 3, 12, 25, 50, 100 and 200 MiB. Figures read from the macOS Activity Monitor are reported as displayed, where 1 GB = 10^9 bytes. Network speeds are in megabits per second (Mbps).
Segment, fragment, chunk	A <i>segment</i> is the unit of plaintext the client encodes independently; in the demonstration profile a segment holds 786,384 bytes. Each segment is encoded into five <i>fragments</i> (also called <i>shards</i>) of 256 KiB each. A fragment at rest on a provider is a <i>chunk</i> , addressed by the SHA-256 hash of its content.
Money	Ledger values are held as integer paise (1 rupee = 100 paise) and displayed in rupees (₹). All payments in the run were in mock mode; no real money moved.
Time	Timestamps are Indian Standard Time (IST, UTC+05:30) on 22 September 2026 unless stated. Durations are written <i>min s</i> or <i>mm:ss</i> .
Provenance	Every measured value carries one of three tags. ^[A] <i>automatic</i> : read from the exported Prometheus time series or printed by one of the system's own programs. ^[M] <i>manual</i> : read from an instrument by an operator (Activity Monitor, TRAI MySpeed, the scribe log, visual inspection). ^[D] <i>derived</i> : computed from two or more ^[A] / ^[M] values, with the arithmetic stated or referenced.
Bounds	$\geq$ and $\leq$ mark a bound rather than a point value — used where a screenshot is known not to coincide with a true peak, or where a value is limited by a histogram bucket or by the 15 s scrape interval.
Unmetered	An <i>unmetered parameter</i> is a quantity the report defines and would measure, but which Run 3 or Run 4 did not capture. It is printed as <i>unmetered</i> and is never estimated or back-filled.
Desks	Providers are identified by their laboratory desk label (for example C13 or 15F). Table 3 maps each desk to its synthetic identity and synthetic network identifier.
Metric IDs	Identifiers of the form D-xxx-nn are stable names for measured quantities; they are listed with their status in Appendix A.

1 Introduction

1.1 Motivation

Demand for stored data keeps growing, and new data-centre capacity is slow to add: it is bounded by land, power and cooling as much as by the price of disks. At the same time, a very large amount of storage already exists at the edge of the network and sits largely unused. A measurement of thousands of corporate desktop computers found enough idle disk space and machine availability to host a serverless file system on the desktops themselves [1], and volunteer-computing systems have for decades harvested the idle cycles of ordinary workstations and home computers at scale [2, 3]. The nine laboratory desktops used in this study illustrate the point on a small scale: each had between 194 and 441 GB of free disk (Table 3).

Turning that idle capacity into a storage service that someone would trust with their files is, however, a hard distributed-systems problem. Desktop computers are switched off, rebooted, reinstalled and abandoned; their owners are not operators and cannot be assumed honest; and the network paths between them cross home routers and institutional firewalls. The literature on peer-to-peer storage has studied each of these difficulties separately — availability under churn [4–6], redundancy and repair [7, 8], confidentiality [9, 10], proofs that data is really stored [11, 12], and incentives [13, 14]. Deployed decentralised storage networks combine them in different ways [15–18].

Vyomanaut is a decentralised cold-storage network designed for India. Its premise is that files which are written once and read rarely — archives, backups, records — can be stored more cheaply and with less concentrated infrastructure on the idle disks of desktops that already exist, provided that the system, rather than any individual desktop, is responsible for keeping each file confidential, intact and available. Providers are paid for the storage they contribute, in proportion to the storage audits they pass.

1.2 The demonstration build

The system is being developed in stages. The first complete stage is a *demonstration build*: a working implementation of every principal mechanism — client-side encryption and erasure coding, direct peer-to-peer transfer, provider onboarding and vetting, placement under a diversity constraint, continuous auditing, reliability scoring, automated repair and escrow accounting — configured by a *demonstration profile* that compresses the system's time constants from days and months to minutes. The compression is what makes the whole lifecycle observable on a handful of real machines in a single afternoon. The same programs run under the profile; the profile changes parameters, not code paths.

1.3 Contributions of this report

This report is the reference description of the demonstration build and the record of its first instrumented field measurement. Specifically, it:

- describes the architecture of the demonstration build and the reasoning behind its principal mechanisms, with reference to the literature on which they rest (Sections 2 and 3);
- documents a ten-machine campus rig and the procedure used to measure it, including the provenance rules under which every number was obtained (Section 4);
- reports measured behaviour for network bring-up, upload and retrieval across a ladder of file sizes, fragment delivery, resource use, auditing, repair, escrow accounting, placement and observability (Section 5); and

- declares, as unmetered parameters, every defined quantity the run did not capture (Appendix A).

1.4 Scope

This is a measurement record from a single-campus, ten-machine rig operated by a small team in one afternoon. It is not a production benchmark, and nothing in it should be read as a performance or durability claim for any configuration other than the one measured. The demonstration profile's parameters (Table 2) are chosen for observability, not for durability or economics; in particular, its erasure-coding parameters, time windows and tariff are demonstration values. The report makes no forward-looking commitments beyond the brief statement in Section 6.1.

1.5 Organisation

Section 2 places the design in the context of prior work. Section 3 describes the demonstration build. Section 4 fixes the hardware, network, software and method against which every later number is measured. Section 5 is the measurement record. Section 6 notes future directions and concludes. The appendices contain the metric register and the raw run log; a glossary and the bibliography close the report.

2 Background and related work

This section summarises the prior work on which the demonstration build draws. It is organised by the problem each body of work addresses rather than by system.

2.1 Decentralised storage networks

Content-addressed peer-to-peer storage was popularised by IPFS, which names data by the hash of its content and locates it through a Kademlia distributed hash table [17, 19]; a large-scale evaluation of the deployed IPFS network characterises its content-retrieval latency and the churn of its peers [20]. IPFS itself provides no guarantee that anyone keeps a given object. Storage networks that do make such a guarantee add three further elements: redundancy across independent providers, a way to check that providers still hold what they were given, and a payment that rewards them for doing so. Filecoin realises these with a blockchain, on-chain proofs of replication and of space-time, and a native token [16]. Storj uses Reed-Solomon erasure coding, a set of coordinating services (*satellites*) that audit providers and account for payment off-chain, and a vetting period during which new providers receive only a small share of data [15]. Tahoe-LAFS, designed around the principle of least authority, encrypts and erasure-codes files on the client so that storage servers learn nothing about the content they hold [10]. A recent systematisation of knowledge compares these and other designs along the axes of incentive, proof and redundancy [18].

Vyomanaunt shares Storj's separation of a coordinating control plane from a peer-to-peer data plane and Tahoe-LAFS's insistence on client-side confidentiality. It uses no blockchain and no token: coordination, audit bookkeeping and payment settlement are performed by a conventional service, and payment is denominated in rupees.

2.2 Desktop computers as a storage substrate

The feasibility of storing data on desktop computers was studied directly by the Farsite project, whose measurements of a large corporate desktop fleet showed ample free space

and sufficient machine availability for a serverless file system [1]. Availability in open peer-to-peer populations is much lower and more variable: measurement studies found short median session times and strong diurnal patterns, and showed that the choice of what counts as “the same host” materially changes the availability one measures [4, 5]. Blake and Rodrigues argued that the bandwidth needed to maintain redundancy under such churn makes high availability, scalable storage and dynamic membership difficult to achieve together [6], and Rhea *et al.* showed how distributed hash tables must adapt their timeouts and maintenance to survive it [21]. Failures of peers are, moreover, not independent: machines that share a network, a building or a power feed fail together, and several analyses show that ignoring this correlation overstates durability substantially [22, 23]. At the level of a single machine, disks and solid-state drives fail partially and silently as well as completely [24, 25], which is why the demonstration’s storage engine verifies the content hash of every fragment it reads. The idea of harvesting idle desktop resources is itself long established in volunteer and cycle-scavenging computing [2, 3].

2.3 Erasure coding and repair

An (n, k) maximum-distance-separable code such as Reed–Solomon stores n fragments of a segment, any k of which reconstruct it, and so tolerates the loss of $n - k$ fragments at a storage overhead of n/k . The cost that dominates in peer-to-peer storage is not the overhead but the bandwidth of *repair*: rebuilding one lost fragment conventionally requires downloading k fragments. Dimakis *et al.* characterised the fundamental trade-off between storage and repair bandwidth and introduced regenerating codes [7]; minimum-storage regenerating codes now exist for all parameters [26], Reed–Solomon codes themselves admit bandwidth-efficient repair [27], and systems work has reduced repair traffic and I/O in practice [28–30]. A complementary line of work reduces repair *frequency*: lazy repair waits until redundancy falls to a threshold before acting, trading a small durability margin for a large saving in bandwidth [8, 31]. Surveys and recent studies cover erasure codes for cold and archival data, demand- and hotness-aware coding, straggler tolerance, tiering inside log-structured stores, encoding efficiency and the persistence of replicated erasure codes [32–39]. The demonstration build uses a conventional Reed–Solomon code with small parameters (Section 3.2); it does not implement regenerating or locally repairable codes.

2.4 Confidentiality of dispersed data

Erasure coding alone does not hide content: a systematic code stores plaintext in its data fragments. Krawczyk’s *secret sharing made short* encrypts the data, disperses the ciphertext with an information-dispersal algorithm and secret-shares only the short key [40]. AONT-RS goes further by removing the key from custody altogether: an all-or-nothing transform makes the package undecipherable unless essentially all of it is present, and the key needed to invert it is recoverable only from the package itself, so that fewer than k fragments reveal nothing practical about the plaintext and no key needs to be stored [9]. Recent work examines the cost of separating encryption from erasure coding in decentralised stores [41]. Information-theoretic security under repair — where an eavesdropper also observes repair traffic — has been studied extensively for regenerating and locally repairable codes [42–51]. The demonstration build follows the AONT-RS construction (Section 3.2).

2.5 Proofs of storage

A provider paid to store data has an incentive to discard it and pretend otherwise. Provable data possession (PDP) and proofs of retrievability (PoR) let a verifier check, with a small challenge and response, that a server still holds a file [11, 12]; compact PoRs made such proofs short and publicly verifiable [52], dynamic PDP supports updates [53], outsourced PoRs delegate the checking [54], and later schemes address multiple replicas and clouds and post-quantum settings [55, 56]. For erasure- and network-coded storage, remote data checking has been combined with repair, including repair performed by the servers themselves [57–59]. The demonstration build uses an interactive challenge–response over individual fragments (Section 3.6).

2.6 Reputation, incentives and payment

BitTorrent showed that simple reciprocal incentives make a swarm robust [13]. Reputation systems such as EigenTrust and PeerTrust aggregate peers’ experiences into trust scores [60, 61], and game-theoretic analyses show that, without a mechanism whose reports cannot be gamed, honest behaviour need not be an equilibrium [62–64]; a systematic review maps the design space of monetary, reputational and service incentives [14]. Deferred compensation — paying part of a worker’s earnings later — is a classical economic device for deterring shirking [65], and it motivates the escrow hold described in Section 3.8. Two studies from outside systems research also informed the design: evidence that opaque, algorithmically managed pay erodes the trust of gig workers [66], and evidence that cryptocurrency wallets remain difficult for ordinary users [67], which favours plain-currency payment over a token.

2.7 Coordination, discovery and storage engines

Highly available stores such as Dynamo replicate state across nodes with tunable read and write quorums [68], and invariant-confluence analysis identifies which operations genuinely require coordination and which can proceed without it [69]. Kademlia provides logarithmic lookup in a distributed hash table [19], and S/Kademlia hardens it against identity and routing attacks [70]. Measurements of decentralised NAT traversal quantify how often peers behind home routers can be reached directly [71]. On the provider, log-structured merge trees give high write throughput [72], and WiscKey’s separation of small keys from large values in an append-only value log removes most of the write amplification that large values would otherwise incur [73] — the property that matters for a store whose values are all 256 KiB fragments.

3 The demonstration build

This section describes the system that was measured. It is a description of behaviour and structure at the level needed to interpret the results, not a specification.

3.1 Roles and the two planes

Three roles participate (Figure 2). A *data owner* runs a command-line client that registers an account, deposits funds, uploads, lists, retrieves and deletes files, and can recover the account on a second device. A *provider* runs a background daemon on a desktop computer that stores fragments, answers audit challenges, sends heartbeats, takes part in repair and accrues earnings. The *coordinator* is a service backed by a PostgreSQL database; it admits and vets providers, maintains the readiness gate, decides placements, issues audit challenges

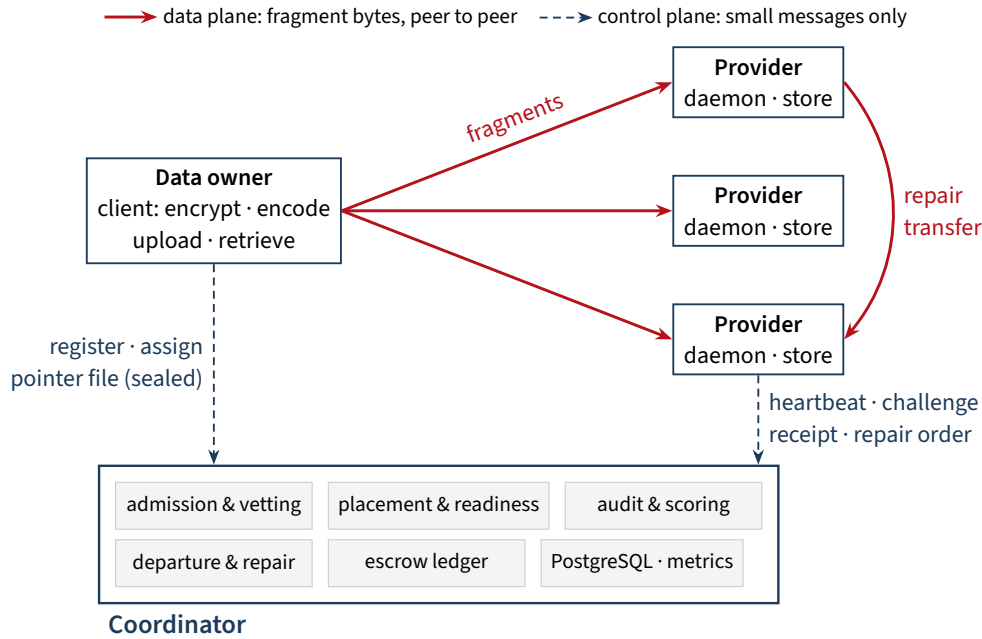


Figure 2. Roles and planes of the demonstration build. Fragments travel only on the data plane (solid); the coordinator exchanges only small control messages (dashed) and never carries or can decrypt file data.

and records their outcomes, detects departures and schedules repair, and keeps the escrow ledger.

The design separates a *data plane*, on which file fragments move directly between owner and providers (and between providers during repair), from a *control plane*, on which the coordinator exchanges small messages — placements, short-lived transfer authorisations, challenges, signed receipts, repair orders and ledger entries — with the other parties. The coordinator never carries a fragment and never holds a key that could decrypt one. This follows the principle, familiar from BitTorrent trackers [13] and from Storj’s satellites [15], that a thin service can tell peers whom to talk to without sitting in the path of the data. In the measured run the coordinator ran as a single replica.

3.2 Encoding pipeline and keys

Encoding happens entirely on the data owner’s device (Figure 3). The client splits a file into segments of 786,384 bytes in the demonstration profile and processes each segment independently. Each segment is first passed through an all-or-nothing transform in the AONT-RS construction [9]: a fresh random key encrypts the segment together with a fixed integrity *canary*, and the key is then folded into the package in a way that can be undone only when the package is complete. The segment cipher is AES-256 in counter mode when the processor offers AES instructions (AES-NI) and ChaCha20 otherwise; the choice changes speed, not security. The package is then encoded with a systematic Reed–Solomon code over $GF(2^8)$ into three data and two parity fragments of 256 KiB each, any three of which reconstruct the segment. Because the transform precedes the code, fewer than three fragments of a segment disclose nothing practical about its content, and no per-file key is ever stored or transmitted. A fragment is named by the SHA-256 hash of its content. Every upload uses fresh randomness, so identical files uploaded twice produce unrelated fragments; the system performs no deduplication.

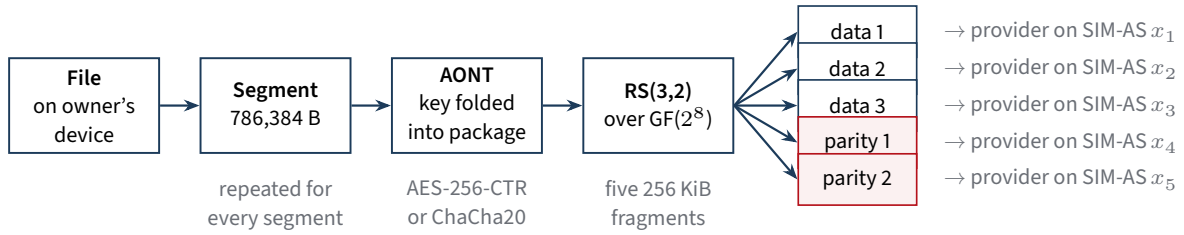


Figure 3. Encoding pipeline of the demonstration profile. Any three of a segment’s five fragments reconstruct it; the placement rule puts its five fragments on five different networks ($x_1, \dots, x_5$ distinct).

What the owner must keep in order to find a file again — which providers hold which fragments, and the encoding parameters — is recorded in a per-file *pointer file*, sealed on the client with the ChaCha20-Poly1305 authenticated cipher and stored at the coordinator as ciphertext the coordinator cannot open. The keys that seal pointer files derive from a single master secret, itself derived from the owner’s passphrase with the memory-hard function Argon2id; purpose-specific keys are derived from it with HKDF-SHA256. At registration the client displays a 24-word BIP-39 mnemonic from which the account can be recovered on another device. Owners and providers hold Ed25519 signing keys, which authenticate their transport connections and sign audit receipts.

3.3 Transport and discovery

Fragments travel over TLS 1.3 connections on TCP in which each endpoint’s certificate is bound to its Ed25519 identity, so that a provider is authenticated by the key it registered rather than by a network address. Peer identifiers follow the libp2p peer-identifier format, although the demonstration build does not use the libp2p library. Fragment addresses can be published in a Kademlia-style distributed hash table maintained by the provider daemons [19]; provider discovery, by contrast, goes through the coordinator. The build contains its own reachability probe, TCP hole punching and relay fallback for peers behind network address translation, but these were not exercised in the measured run, where every machine was reachable through the ZeroTier overlay (Section 4.3).

3.4 Admission, vetting and the readiness gate

A provider is admitted with a phone number confirmed by a one-time code (delivered through a log file rather than SMS in the demonstration), a declared storage allocation and a payout account. A newly admitted provider enters a *vetting* period in which it holds only a synthetic fragment generated by the coordinator and is audited on it like any other fragment; it becomes **ACTIVE** after a minimum number of audit passes over a minimum time. Holding back new providers in this way follows the practice of deployed networks [15] and the logic of deferred compensation [65]. The network accepts uploads only when a *readiness gate* is satisfied: enough active vetted providers, on enough distinct networks, in enough metropolitan regions, with enough relay nodes and enough payout accounts past their cooling period (Table 2).

3.5 Placement

Each network a provider connects through is identified by its autonomous system number (ASN). The placement rule caps the fraction of any one segment’s fragments that may sit on a single ASN, so that the failure of one network operator cannot take out a segment; with five fragments and a cap of 0.2, every fragment of a segment must sit on a different ASN. Among

eligible providers, placement favours those with higher reliability scores. Correlated failure is the main reason for the cap [22, 23]. Because all providers in the rig shared one campus network, each was assigned its own *synthetic* ASN; the cap was therefore exercised as logic rather than as protection against a real correlated failure.

3.6 Auditing and scoring

Every stored fragment is audited once per audit period. The coordinator issues a challenge carrying a fresh nonce generated at the moment of challenge, so that it cannot be predicted or precomputed; the provider must read the fragment and return a response bound to both the fragment's content and the nonce, signed with its Ed25519 key, within a deadline scaled to the fragment size and the provider's declared bandwidth. Outcomes are **PASS**, **FAIL** (the provider reports that it cannot produce the fragment, or the response is invalid) and **TIME-OUT**. The design avoids peer-reported audits in favour of a single, narrowly scoped referee because incentive analyses show that unrefereed peer reporting need not sustain honest behaviour [62, 63]. The coordinator checks the signature, nonce and timeliness of each response and records the outcome as a signed receipt; it never receives the fragment itself. The design is informed by the literature on possession and retrievability proofs [12, 52].

Each provider's reliability score is computed from its audit outcomes over three sliding windows (short, medium and long), in the spirit of history-based reputation [60, 61]. Scores feed placement.

3.7 Departure and repair

Providers send heartbeats at a fixed interval with random jitter. A provider that is silent for longer than the departure threshold, or that announces its departure, is marked **DEPARTED**, and every fragment it held becomes a repair job. A repair reconstructs the segment from any three surviving fragments and writes the replacement fragment to a new provider chosen under the same diversity cap; fragment data moves provider to provider on the data plane. Repair can be deferred until redundancy falls to a threshold (lazy repair [8, 31]); departures are repaired immediately.

3.8 Escrow and payment

The data owner deposits funds into escrow. At a fixed charge interval the coordinator charges the owner for stored files at the profile's tariff, computed on the original file size; at a release interval it computes provider earnings in proportion to audits passed and releases them after a hold window. All amounts are integer paise, and balances are never stored directly: they are derived by summing append-only ledger events, the same pattern as a positive-negative counter in conflict-free replicated data types. The build contains an adapter for the Razor-pay payment gateway; the measured run used the *mock* payment mode, in which no money moves.

3.9 Provider storage engine

Provider daemons store fragments in a key-value store that separates small keys from the large fragment values, in the manner of WiscKey [73]: on macOS and Linux a RocksDB index over an append-only value log, and on Windows — the platform of every provider in the run — BadgerDB, which uses the same separation. A single writer serialises appends. Every read verifies that the SHA-256 hash of the fragment matches its content address, so that silent

corruption is detected rather than served [24]. A daemon raises a *RAM-constrained* flag if memory falls below what its store requires.

3.10 Observability

The coordinator and every provider daemon expose Prometheus metrics, scraped every 15 s. A Grafana dashboard of 21 panels displays them, including three panels added for this measurement: assignment versus confirmed delivery, the cumulative assignment–delivery gap, and audit pass and fail rates. An operator console, a terminal user interface refreshed every 5 s, shows the readiness gate, per-network fragment counts, the provider fleet, repair, audit, escrow and an event feed (Figure 5).

3.11 The demonstration profile

All time constants and thresholds are held in a single configuration structure, the *NetworkProfile*, which each program prints at start-up. Table 2 lists the values printed by the running programs throughout the measured run; every duration and threshold in Section 5 is judged against it. The fragment size of 256 KiB is a wire-format invariant, not a demonstration value.

Table 2. Demonstration profile, as printed by the running programs ^[A]

Parameter	Value	Parameter	Value
Data / parity / total fragments	3 / 2 / 5	Heartbeat interval / jitter	30 s / 5 s
Fragment size	262,144 B	Departure threshold	10 min
Segment plaintext	786,384 B	Audit period	2 min
Repair threshold above k	1 fragment	Vetting: min. passes / duration	5 / 5 min
Readiness: active providers	≥ 5	Score windows (short/med./long)	2 / 6 / 20 min
Readiness: distinct ASNs	≥ 5	DHT republish / expiry	2 / 4 min
Readiness: metro regions / relays	$\geq 1 / \geq 0$	Charge / release interval	1 min 30 s / 2 min
Readiness: cooled payout accounts	≥ 5	Escrow hold window	1 min
ASN cap per segment	$0.2 (\lfloor 5 \times 0.2 \rfloor = 1)$	Tariff (demonstration value)	₹1,000 per GiB-month
		Payment mode	mock

4 Experimental setup and method

4.1 Machines

Table 3 was recorded on each machine before the rig came up. Two provider desks, 15E and 15F, have half the memory of the other seven (4 GB against 8 GB); Section 5.1 examines whether memory affected onboarding. The coordinator laptop also hosted the data-owner client and the whole observability stack.

Table 3. Machine inventory, recorded at rig start ^[M]

Role	Desk	OS	Cores	RAM	Disk free / total	Identity / ASN
Coord. + owner	MacBook Air M2	macOS 27.0	8	8 GB	59 / 245 GB	—
Provider 1	C13	Win 11 Pro	4	8 GB	369 / 416 GB	...890 / AS1
Provider 2	C26	Win 11 Pro	4	8 GB	195 / 243 GB	...891 / AS2
Provider 3	C37	Win 11 Pro	4	8 GB	397 / 440 GB	...892 / AS3
Provider 4	C21	Win 11 Pro	4	8 GB	202 / 243 GB	...893 / AS4
Provider 5	C36	Win 11 Pro	4	8 GB	390 / 440 GB	...894 / AS5
Provider 6	15F	Win 11 Pro	4	4 GB	424 / 464 GB	...895 / AS6
Provider 7	15E	Win 11 Pro	4	4 GB	441 / 464 GB	...896 / AS7
Provider 8	C16	Win 11 Pro	4	8 GB	399 / 440 GB	...897 / AS8
Provider 9	C32	Win 11 Pro	4	8 GB	194 / 243 GB	...898 / AS9

Every provider ran Windows build 22621 on an Intel Core i5 processor of unrecorded generation (an unmeasured parameter). Identities are the synthetic demonstration numbers +9112345678xx; ASNs are synthetic, one per provider (SIM-AS1...SIM-AS9).

4.2 Network

Table 4 was measured with the TRAI MySpeed application at the start of the run, before any provider joined. An end-of-run pass was obtained only for the coordinator, because the campus wired network failed before the provider desks could be re-measured (Section 5.6).

Table 4. Network characterisation (TRAJ MySpeed), start-of-run pass ^[M]

Desk	Uplink type	Down (Mbps)	Up (Mbps)	Latency (ms)	Jitter (ms)
MacBook Air (coordinator, owner)	Mobile hotspot	202.7	36.2	32	4
C13	Campus Ethernet	562.5	109.5	42	5
C26	Campus Ethernet	552.5	108.3	21	3
C37	Campus Ethernet	552.5	108.3	21	3
C21	Campus Ethernet	563.4	105.6	20	2
C36	Campus Ethernet	541.8	115.0	25	6
15F	Campus Ethernet	551.3	121.2	15	2
15E	Campus Ethernet	550.6	131.7	17	3
C16	Campus Ethernet	560.3	93.6	21	3
C32	Campus Ethernet	551.9	103.4	24	2

End-of-run pass (coordinator only): 243.0 Mbps down, 54.5 Mbps up; latency and jitter unmeasured. Packet loss was 0.0% on every pass. The owner's hotspot uplink (≈ 36 Mbps) is about one third of any provider's (≈ 94 – 132 Mbps); this asymmetry bounds upload throughput (Section 5.2).

4.3 Overlay

All ten machines joined a single ZeroTier virtual network. Every provider connected over campus Ethernet and the coordinator over its mobile hotspot. Network address translation traversal in this rig was therefore performed by ZeroTier, not by Vyomanaunt; the system's transport ran over an already-established overlay, and the traversal mechanisms described in Section 3.3 were not exercised.

4.4 Software

Table 5. Software under test and instrumentation

Component	Version / configuration
Language and toolchain	Go 1.26.2 on every machine
Overlay	ZeroTier 1.16.2 on every machine
Coordinator store	PostgreSQL, schema generated for the demonstration profile at start-up
Provider stores	BadgerDB on the nine Windows provider daemons
Metrics	Prometheus 2.53 and Grafana 11.1 in Docker on the coordinator laptop; 15 s scrape interval
Operator console	Terminal user interface, demonstration mode, 5 s refresh
Client-side instruments	macOS Activity Monitor (memory, CPU, network); TRAI MySpeed (network)

4.5 Runs

The demonstration build was exercised in four runs on the same day (Table 6). This report presents Run 3, the only run with a clean, file-size-indexed ladder and the assignment-delivery instrumentation, together with the part of Run 4 that completed. Runs 1 and 2 were exploratory; their values are not reported here because they were not captured under the provenance rules of Section 4.7.

Table 6. Runs on 22 September 2026

Run	Providers	Coverage	Status
1 (morning)	5→9	Bring-up, small-file upload and retrieval, one departure and repair cycle	Exploratory; not reported
2 (evening)	9	Large-file exercise (≈ 730 MB); informed the instrumentation added for Run 3	Exploratory; not reported
3	9	Instrumented bring-up; upload and retrieval ladder 3–200 MiB; steady-state auditing	Reported in full
4	9	Announced departure and repair; churn phases	Interrupted at 16:50 by campus network failure; reported as far as it progressed

4.6 Workload and procedure

The coordinator was started first; providers were then brought in one desk at a time by an operator, each joining with a one-time code. Once all nine providers were active, the data owner registered, deposited ₹10,000.00 in mock mode, and uploaded six synthetic files of random, incompressible bytes in ascending size: 3, 12, 25, 50, 100 and 200 MiB (the *ladder*). Each upload was followed by a file listing, a retrieval, an operator placement listing and a balance check. Run 4 then began with provider C32 announcing its departure.

4.7 Measurement provenance and aggregation

Three classes of value are used and tagged throughout ([Notation and conventions](#)). The Prometheus database was exported at the end of the day and replayed offline to produce every time series and every automatic value in Section 5; charts are redrawn from that data rather than reproduced as screenshots wherever a real series exists. Counter-based values are computed as exact differences between counter samples at the stated boundaries, not by rate extrapolation. Terminal outputs were transcribed from screenshots and cross-checked against the series wherever both exist; for example, the coordinator's assignment counter increased by exactly the fragment count reported by the operator's placement listing for every upload. Client-side (data-owner) resource figures come from Activity Monitor readings. Windows daemons report the process working set as resident memory, which the operating system can trim; their Go heap is reported alongside for that reason.

Audit pass rates are never averaged across system states: a pass rate measured while a large upload is in flight describes a different state from one measured at rest (Section 5.5). Upload throughput is reported per file size only. No value is averaged across runs.

4.8 Measurement conditions

- **One campus, one afternoon, one team.** Every provider shared one wired network; results describe that environment.
- **Synthetic ASNs.** Each provider was assigned its own synthetic ASN, so the diversity cap was exercised as logic, not as protection.
- **Asymmetric uplinks.** The owner sat behind a 36 Mbps hotspot while providers had ≈ 100 Mbps; upload figures are bounded by the former.
- **Co-located roles.** Coordinator, data-owner client, PostgreSQL, Prometheus and Grafana shared one 8 GB laptop that was also running unrelated applications; client resource figures include that contention.
- **Overlay dependency.** All traffic crossed ZeroTier, which adds encapsulation overhead and performed NAT traversal on the system's behalf.
- **Small n .** One run per file size; no confidence intervals are claimed.
- **Scrape resolution.** Times derived from Prometheus are bounded to ± 15 s.
- **Interruption.** A failure of the campus wired network at 16:50 made every provider unreachable while the coordinator remained up; Run 4 ended at that point (Section 5.6).

5 Results

Each subsection pairs a short narrative with a metric table. Metric identifiers are stable ([Appendix A](#)). Values that the run did not capture are printed as *unmetered*.

5.1 Network bring-up

The coordinator started at 14:06:02. Desks were brought in one at a time; the first joined at 14:13:45 and the ninth at 14:32:24, a mean interval of 2 min 20 s set by the manual onboarding procedure rather than by the system. The readiness gate (five vetted providers on five distinct ASNs) was met at about 14:41:18 according to the console's event feed and displayed **READY** at 14:41:57 (Figure 5a). All nine providers were **ACTIVE** at 15:03:37 (Figure 5b).

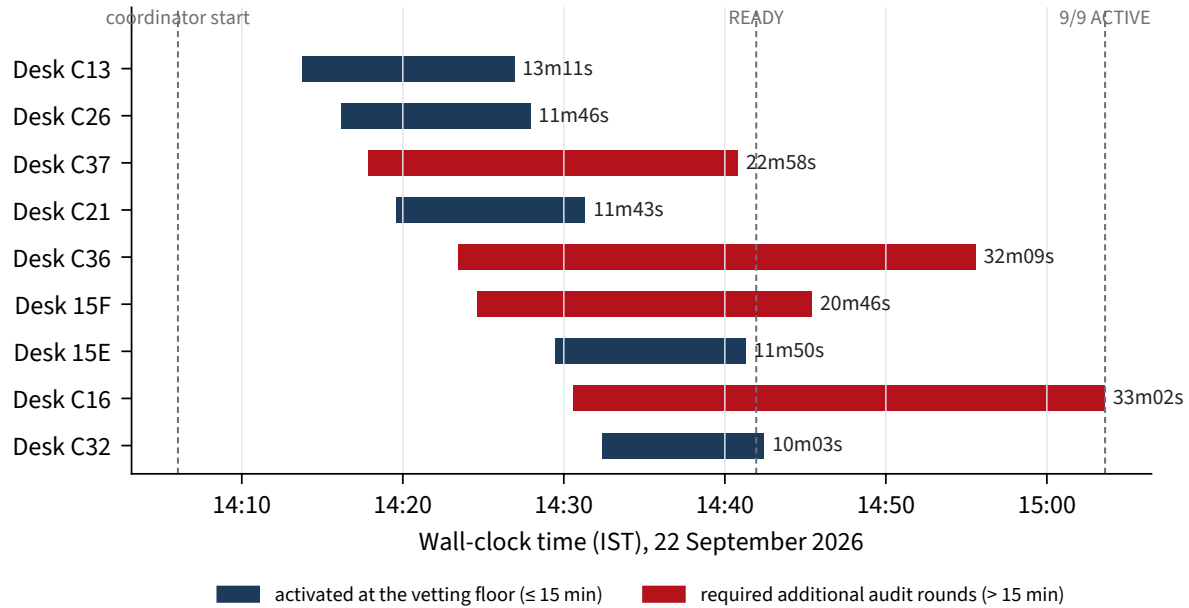


Figure 4. Per-provider onboarding, from daemon start to ACTIVE (Run 3). Navy bars activated at the vetting floor; red bars needed additional audit rounds.

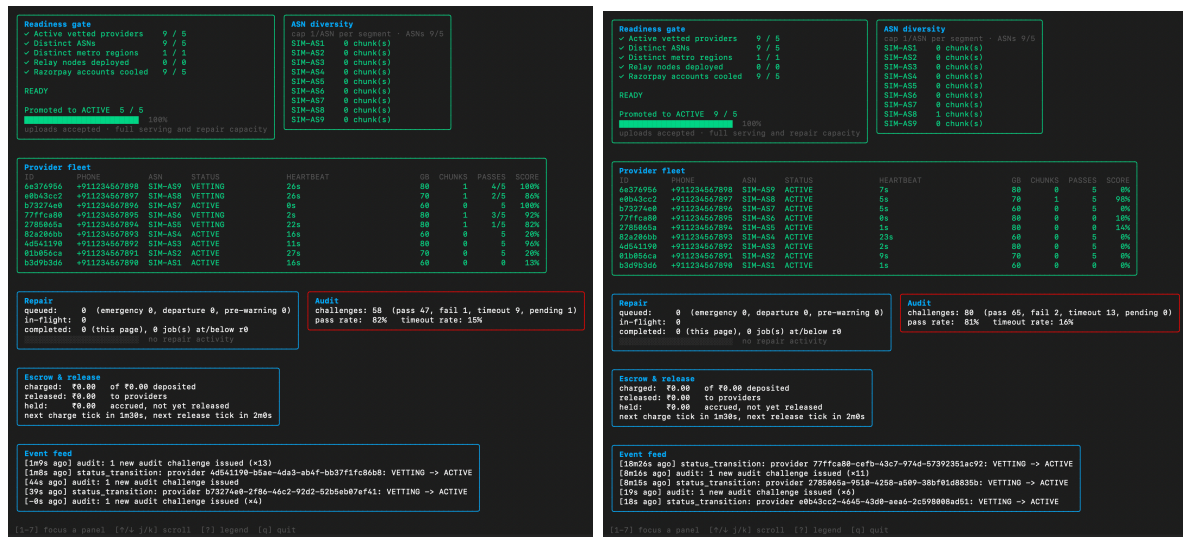


Figure 5. Operator console at the two bring-up milestones. The SCORE column in (b) is discussed in the text.

With a two-minute audit period and a requirement of five passes over at least five minutes, the practical floor for vetting is about ten minutes. Four providers (C32, C21, C26, 15E) activated within twelve minutes of joining, that is, at the floor; C13 took thirteen minutes. Four took 21–33 minutes (Figure 4). The slow group is explained by the audit outcomes during vetting: of the 81 audits resolved before the first upload, 16.0 % timed out and 3.7 % failed (Table 12), and every non-pass delays the fifth pass by a full audit period. Memory is not the explanation: of the two 4 GB desks, 15E activated at the floor (11:50) and 15F took 20:46, while the slowest desk, C16, has 8 GB.

Scribe-log correction

The written log records C36 as **ACTIVE** at 15:03:37 and C16 at 14:55:35. The console event feed captured at 15:03:37 (Figure 5b) shows C16 transitioning 18 s earlier and C36 about eight minutes earlier, and the vetting-period audit series agree. The two entries were transposed in the log; this report uses the automatically recorded order.

Scores immediately after activation. At full activation, eight of the nine providers showed a reliability score of 0–14 %; C16, the last to activate, stood at 98 %. The scores reflect the audit windows rather than provider behaviour: five desks (C13, C26, C21, 15E, C32) received no audit challenge between activation and the first upload at 15:16, because an active provider holding no file fragments has nothing to be audited on, and the 2-, 6- and 20-minute score windows therefore held no evidence. Scores rose as file fragments arrived.

Table 7. Network bring-up metrics (Run 3)

ID	Metric	Value	Remark
D-NET-01	Coordinator start → ready	35 min 55 s ^[D]	Gate met ≈14:41:18; dominated by manual join cadence
D-NET-02	Coordinator start → 100 % active	57 min 35 s ^[D]	Last join 14:32:24
D-NET-03	Join → active, $n = 9$	mean 18:36; median 13:11; min 10:03 (C32); max 33:02 (C16) ^[D]	Floor ≈10 min by construction
D-NET-04	One-time code issue → use	<i>unmetered</i>	Not logged per desk
D-NET-05	Overlay authorisation → address	<i>unmetered</i>	Not logged per desk
D-NET-06	Join → metrics target healthy	<i>unmetered</i>	Not logged per desk
D-NET-07	Operator join cadence	mean 2 min 20 s ^[D]	Manual procedure, not a system property
D-NET-08	Active providers with score ≤ 15 % at 100 % active	8 of 9 ^[A]	Empty score window after activation
D-NET-09	Providers with no audit between activation and first upload	5 of 9 ^[A]	No fragments, so no audits

5.2 Upload and retrieval across the file-size ladder

Six synthetic, incompressible files were uploaded in ascending size, each followed by a listing, a retrieval, a placement listing and a balance check. Table 8 is the core record; Figure 1 plots it.

Throughput is network-bound up to 50 MiB. The owner’s hotspot uplink of 36.2 Mbps carries at most 4.53 MB/s (4.32 MiB/s); after the 5/3 expansion of erasure coding that is 2.59 MiB/s of file data. The 50 MiB upload reached 2.1 MiB/s, 81 % of that ceiling. Small files are slower per byte because fixed per-file costs — unlocking the local identity, obtaining placements and registering the pointer file — dominate. At 100 MiB throughput was 1.5 MiB/s. The 200 MiB

Table 8. File-size ladder (Run 3)

File	Segments	Fragments	Upload ^[A]	Retrieval ^[A]	Upload start	Gap peak / converged after ^[A]
3 MiB	5	25	2.7 s (1.1 MiB/s)	5.3 s (0.6 MiB/s)	15:15:57	17 / ≤ 15 s
12 MiB	17	85	7.1 s (1.7)	13.4 s (0.9)	15:29:35	49 / ≤ 15 s
25 MiB	34	170	14.1 s (1.8)	29.4 s (0.9)	15:37:32	140 / 30 s
50 MiB	67	335	24.3 s (2.1)	51.0 s (1.0)	15:46:36	259 / 30 s
100 MiB	134	670	64.7 s (1.5)	96.3 s (1.0)	15:51:26	653 / 75 s
200 MiB	267	1,335	43 min 34 s ^[D] (≈ 0.08)	not attempted	15:56:51	1,335 / 43 min

Segment and fragment counts are the coordinator's, and agree exactly with the increments of its assignment counter. The 200 MiB duration is wall-clock time from start to completion of the second resumption (15:56:51 to 16:40:25); the client's own printed duration covers the final resumption only.

file was stored in full after two resumptions of the transfer, 43 min 34 s after it started; the resume mechanism carried the upload from 1,280 to 1,323 and then to all 1,335 acknowledged fragments.

Retrieval scales linearly. Retrieval took 0.72 s per segment at 100 MiB and held 0.9–1.0 MiB/s from 12 to 100 MiB, far below the owner's 202 Mbps downlink. The client fetches the fragments of one segment in parallel but processes segments one after another, so retrieval time is bound by per-segment latency.

Wire amplification. Activity Monitor's system-wide *data sent* counter rose by 3.6–4.0 times the file size for each of the four uploads (12–100 MiB) that the counter could resolve (Table 9). The counter includes traffic on both the ZeroTier virtual interface and the physical interface that carries it encapsulated, so each byte is plausibly counted twice; on that assumption the per-hop amplification is 1.8–2.0 $\times$, against 1.67 $\times$ from erasure coding alone. The ZeroTier process itself sent 208 MB between the 50 and 100 MiB captures, which covers the 100 MiB upload (175 MB of fragments) plus audit and retrieval traffic. The double-counting assumption was not tested with an interface-level counter and remains an assumption.

Table 9. System-wide bytes sent per upload (Activity Monitor, 0.01 GB resolution) ^[M]

File	Before (GB)	After (GB)	Δ (MB)	File (MB)	Ratio
3 MiB	1.10	1.11	10	3.1	below resolution
12 MiB	1.13	1.18	50	12.6	3.97
25 MiB	1.21	1.31	100	26.2	3.81
50 MiB	1.38	1.57	190	52.4	3.62
100 MiB	1.68	2.08	400	104.9	3.81
200 MiB	2.30	<i>unmetered</i>	—	209.7	<i>unmetered</i>

Each Δ is a difference of two readings displayed to 0.01 GB, so it is uncertain by up to about ± 10 MB; the resulting ratio is uncertain by up to roughly ± 20 % at 12 MiB, ± 10 % at 25 MiB, ± 5 % at 50 MiB and ± 3 % at 100 MiB.

5.3 Assignment and confirmed delivery

The coordinator records a fragment as *assigned* when it commits a placement; a provider records it as *stored* when the fragment is durably written. The two are independent signals,

and the instrumentation added for this run compares them directly (Figure 6). After every upload, including the 200 MiB upload, the difference returned to exactly zero: every fragment the coordinator believed placed was eventually confirmed stored (Table 21 in Appendix B). The gap is therefore a measure of delivery latency, not of lost data. The comparison must exclude the synthetic vetting fragments, which increment the providers' stored counters (nine at 15:10, one per provider) but not the coordinator's assignment counter; Figure 6 does so. Likewise, the gap reads slightly negative while repair traffic stores fragments outside the upload path.

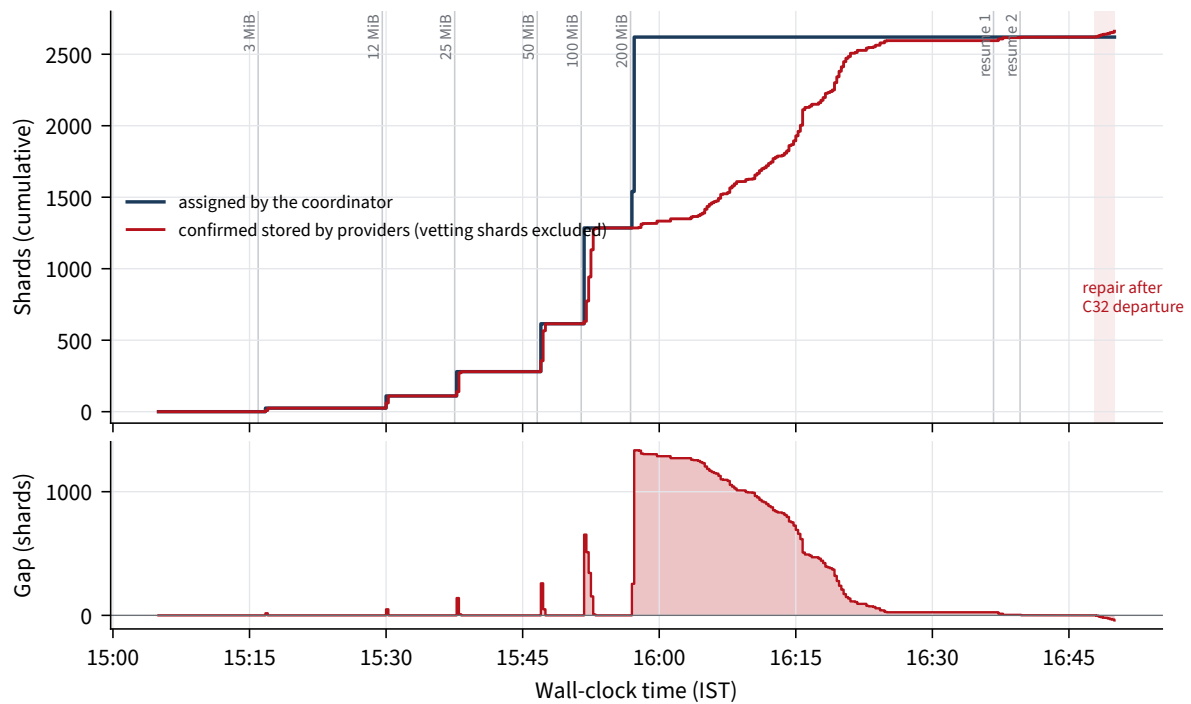


Figure 6. Top: cumulative fragments assigned by the coordinator and confirmed stored by providers. Bottom: their difference. Redrawn from the exported Prometheus series at 15 s resolution; series end at 16:50:00, the last scrape before the campus network failed.

Table 10. Upload and retrieval metrics (Run 3)

ID	Metric	Value	Remark
D-UPL-01	Upload duration per rung	2.7 / 7.1 / 14.1 / 24.3 / 64.7 s (3–100 MiB); 43:34 (200 MiB) ^[A] / ^[D]	Table 8
D-UPL-02	Upload throughput	1.1 / 1.7 / 1.8 / 2.1 / 1.5 / 0.08 MiB/s ^[A]	81 % of uplink ceiling at 50 MiB
D-UPL-03	Segments / fragments	5/25, 17/85, 34/170, 67/335, 134/670, 267/1,335 ^[A]	Coordinator record
D-UPL-04	Wire amplification	3.6–4.0× system-wide; ≈1.8–2.0× per hop ^[D]	Assumes overlay double count
D-UPL-05	Interruptions, 200 MiB	first attempt paused at 1,280 of 1,335 acknowledged ^[M]	Resumed twice
D-UPL-06	Largest file uploaded	200 MiB (2 resumptions); 100 MiB uninterrupted ^[D]	Largest rung run
D-UPL-07	Assignment–delivery gap	peak equals fragment count of each file; returns to 0 after every rung ^[A]	43 min to converge at 200 MiB
D-UPL-08	Resume after interruption	1,280 → 1,323 → 1,335 over two resumptions ^[M]	Completed
D-RET-01	Retrieval duration / rate	5.3 / 13.4 / 29.4 / 51.0 / 96.3 s; 0.6–1.0 MiB/s ^[A]	Segments fetched sequentially
D-RET-02	Byte-exactness of retrieved files (SHA-256)	<i>unmetered</i>	—
D-RET-03	Retrieval with providers absent	<i>unmetered</i>	Run 4 phase not reached
D-RET-04	Recovery on a second device	<i>unmetered</i>	Run 4 phase not reached
D-RET-05	Format fidelity with real media files	<i>unmetered</i>	—

5.4 Resource behaviour

Coordinator host. The coordinator laptop also ran the data-owner client, PostgreSQL, the observability stack in Docker (1.62 GB) and unrelated desktop applications. With the fleet idle it used 6.77 of 8.00 GB of memory with 3.88 GB of swap. Client-side resource figures were read from Activity Monitor at the moments listed in Table 11 and include this shared load.

Coordinator and providers are light. The coordinator process peaked at 43 MB resident and used 635 s of CPU over 2 h 45 min, about 6 % of one core. Provider daemons peaked at 143–223 MB working set, with a Go heap of about 178 MB and 31–32 goroutines each (Figure 7). No daemon ever raised its RAM-constrained flag, including the two 4 GB desks.

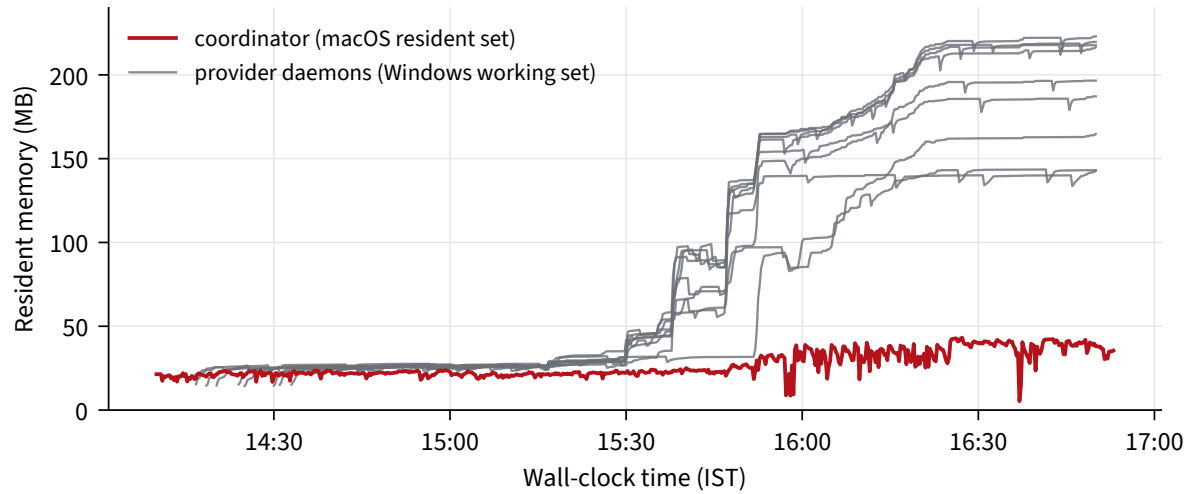


Figure 7. Resident memory of the coordinator and of the nine provider daemons (Run 3).

Table 11. Resource metrics (Run 3)

ID	Metric	Value	Remark
D-RES-01	Client memory at capture	50 MiB: 839 MB; 200 MiB: 7.36 GB ^[M]	Shared host; other rungs not in frame
D-RES-02	Client memory ÷ file size	16× (50 MiB); 35× (200 MiB) ^[D]	At the same captures
D-RES-03	Host swap in use	idle: 3.88 GB; during 200 MiB: up to 10.31 GB ^[M]	Shared host
D-RES-04	Host CPU during 200 MiB	kernel 158 %, client 55 % ^[M]	Activity Monitor
D-RES-05	Coordinator peak resident memory / CPU	43 MB; 635 s CPU in 2 h 45 min ^[A]	Excludes PostgreSQL
D-RES-06	Provider daemon peak memory	143–223 MB working set; heap ≈178 MB ^[A]	Windows working set
D-RES-07	Provider disk footprint	<i>unmetered</i>	—
D-RES-08	RAM-constrained occurrences	0 ^[A]	Includes 4 GB desks
D-RES-09	Value-log append latency, p95	1.75–2.80 ms (8 GB desks); 4.51–4.60 ms (4 GB desks) ^[A]	All under 5 ms

5.5 Auditing

By 16:50:00, the last scrape before the campus network failed, the coordinator had issued 73,833 audit challenges and resolved 73,822 (Figure 8). At rest the audit loop resolved about 21 challenges per second, one per stored fragment per two-minute period. Pass rates differ sharply by system state, which is why they are reported per window and never averaged (Table 12).

Table 12. Audit outcomes by system state (exact differences of the result counters) ^[A]

Window	State	Results	Pass	Fail	Timeout
14:10–15:15	vetting only, no files	81	80.2 %	3.7 %	16.0 %
15:15–15:57	ladder, 3–100 MiB	6,434	94.0 %	1.7 %	4.4 %
15:57–16:41	200 MiB upload in flight	55,755	77.9 %	17.3 %	4.8 %
16:41–16:50	at rest after convergence (repair begins 16:47:45)	11,552	98.1 %	0.0 %	1.9 %
14:10–16:50	all states combined (reconciliation only)	73,822	82.4 %	13.3 %	4.3 %

The last row sums the four windows so that the total can be reconciled with the counters; it is not a system state and no pass rate should be quoted from it (Section 4.7). The 568 results recorded between 16:50 and 16:52, after every provider had become unreachable, were 87 % timeouts and are excluded from every window.

Failures were concentrated in the window in which the 200 MiB file’s fragments were being delivered (9,672 in 44 minutes, coinciding with the assignment–delivery gap of Figure 6), and fell to zero once every assigned fragment was confirmed stored. Timeouts held at 2–5 % outside vetting. Provider audit-response latency at the 95th percentile was at or below 10 ms on every desk — the lowest histogram bucket — and no provider ever reported a content-hash failure.

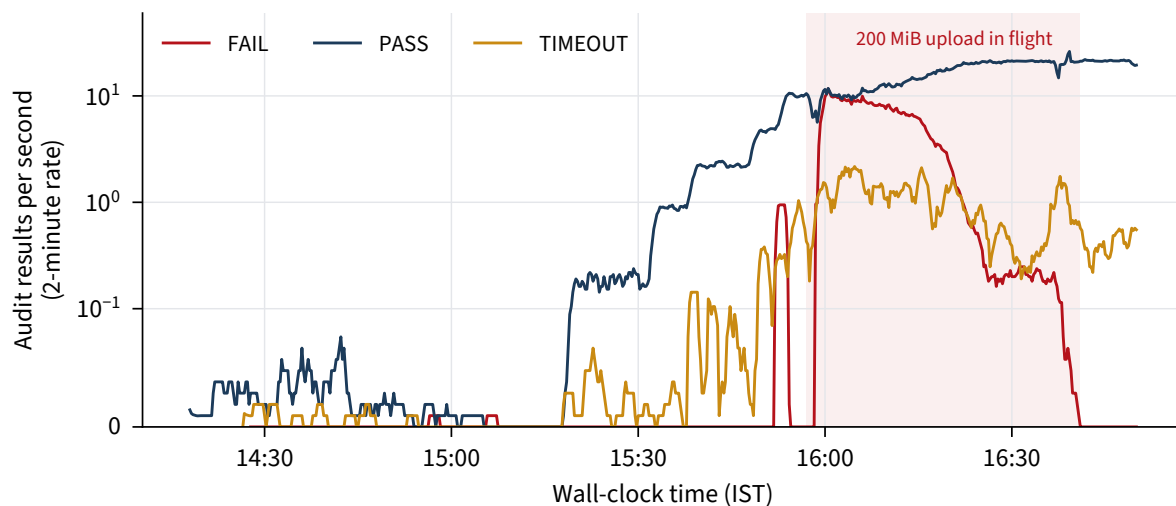


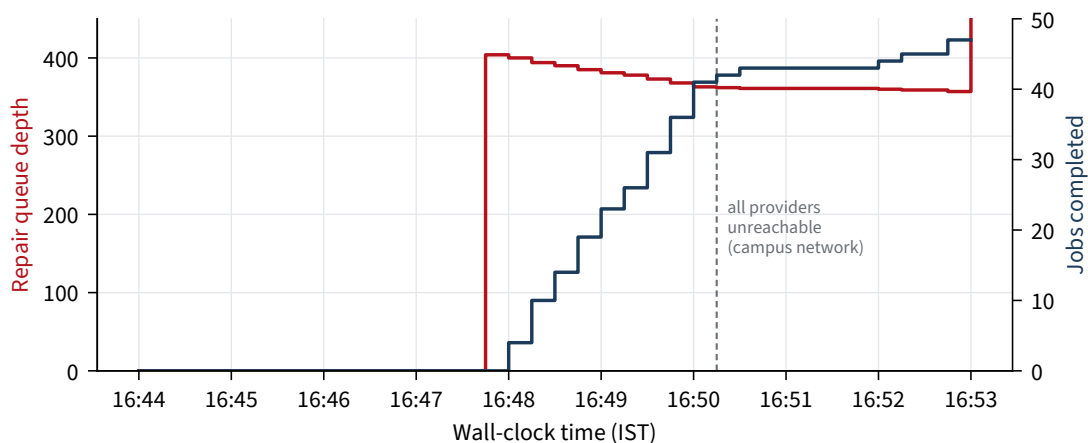
Figure 8. Audit results per second by outcome (two-minute rate, symmetric-log scale), 14:10–16:50.

Table 13. Audit metrics (Run 3)

ID	Metric	Value	Remark
D-AUD-01	Challenges issued	73,833 to 16:50; $\approx 21/s$ at rest ^[A]	One per fragment per audit period
D-AUD-02	Pass rate, steady state	94.0 % (ladder); 98.1 % (at rest) ^[A]	Never averaged with the stress window
D-AUD-03	Timeout rate, steady state	4.4 % (ladder); 1.9 % (at rest) ^[A]	16.0 % during vetting
D-AUD-04	Response latency, p95	≤ 10 ms on all nine desks ^[A]	Bounded by histogram bucket
D-AUD-05	Pass rate under upload load	77.9 % (fail 17.3 %) ^[A]	Coincides with delivery window
D-AUD-06	Content-hash failures	0 ^[A]	All desks

5.6 Repair (Run 4, partial)

Run 4 began with provider C32 announcing its departure. C32 held 405 file fragments; at 16:47:45 the repair queue rose to 404 jobs (Figure 9). Jobs then completed at about 16 per minute – 43 by 16:50:30 – and the rebuilt fragments landed on other providers: 41 new stores by the last provider scrape at 16:50:00, eleven of them on 15F and seven on 15E, two of the three least-loaded desks. Repair-job duration had a median of 68.8 s and a 95th percentile of 244.7 s ($n = 43$, interpolated from the histogram). At 16:50:15 a failure of the campus wired network made every provider unreachable while the coordinator remained up, and Run 4 ended with 361 jobs queued (Figure 9); the console capture of Figure 10 shows 402 queued at an earlier moment. Quantities that depended on the remainder of Run 4 are unmetered (Table 14).

**Figure 9.** Repair queue depth and completed repair jobs after C32 announced its departure.

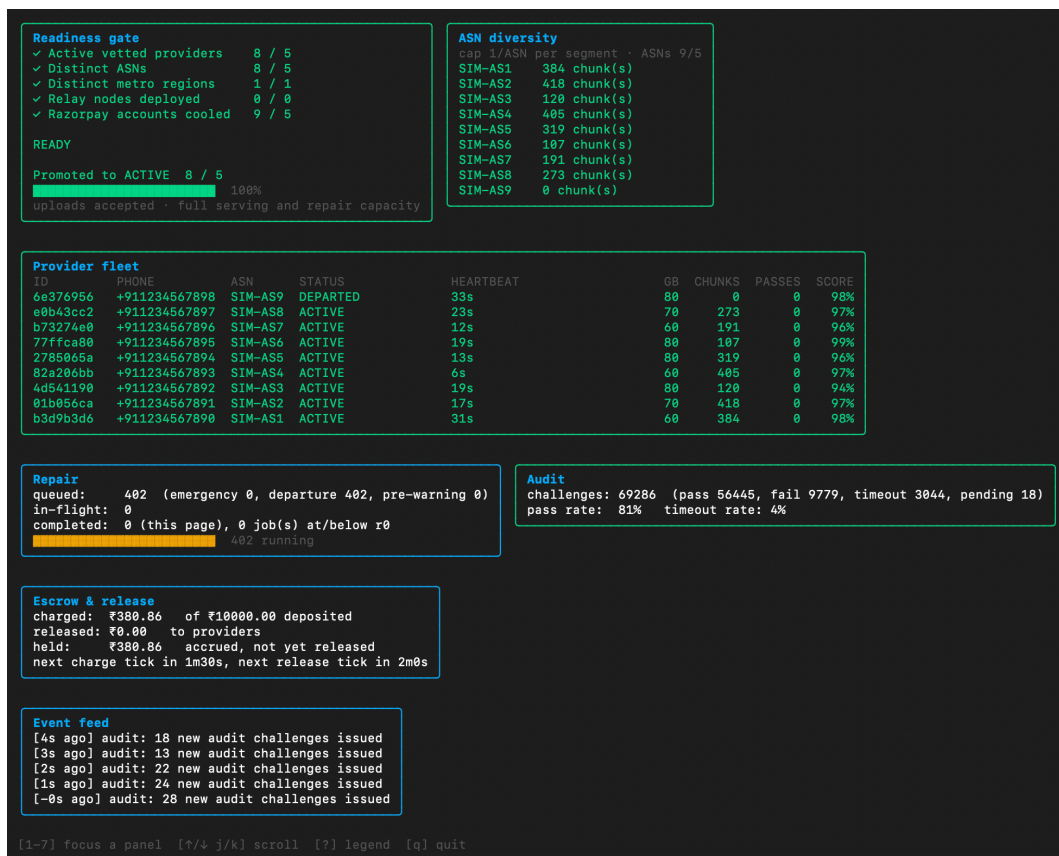


Figure 10. Operator console shortly after C32 (SIM-AS9) announced its departure: the provider is **DEPARTED** with no fragments attributed, 402 departure-repair jobs are queued, and the escrow panel shows ₹380.86 charged against the ₹10,000.00 deposit. Console counters are read from the coordinator’s database at the moment of capture and therefore differ slightly from the Prometheus values quoted elsewhere.

Table 14. Repair and durability metrics (Run 4, partial)

ID	Metric	Value	Remark
D-REP-01	Departure detection latency	<i>unmetered</i>	Announcement time not logged
D-REP-02	Repair queue peak	404 jobs ^[A]	One departing provider
D-REP-03	Time to drain queue	<i>unmetered</i> (361 queued at 16:50:30) ^[A]	Run 4 ended
D-REP-04	Job duration p50 / p95	68.8 s / 244.7 s ($n = 43$) ^[A]	Histogram interpolation
D-REP-05	Completed ÷ queued	43 / 404 at 16:50:30 ^[A]	Run 4 ended
D-REP-06	Post-repair diversity-cap compliance	<i>unmetered</i>	Not reached
D-REP-07	Repair when replacement shares an ASN	<i>unmetered</i>	Not reached
D-DUR-01	Retrieval with providers dead	<i>unmetered</i>	Not reached
D-DUR-02	Fragments unrecoverable	<i>unmetered</i>	Not reached
D-DUR-03	Integrity after churn	<i>unmetered</i>	Not reached

5.7 Escrow and payment accuracy

The data owner deposited ₹10,000.00 in mock mode and checked the balance after every upload (Table 15). At each checkpoint the reserved amount equalled the sum of the files' monthly costs exactly, and the balance equalled the deposit less that sum exactly: zero paise of drift. The monthly cost is computed on the original file size at the demonstration tariff (for example, 200 MiB → ₹195.31).

Table 15. Owner ledger checkpoints (client balance output) ^[A]

Time	After	Balance (₹)	Reserved (₹)	Available (₹)	Σ monthly cost (₹)
15:11:41	deposit	10,000.00	0.00	10,000.00	0.00
15:21:33	3 MiB	9,997.07	2.93	9,994.14	2.93
15:33:54	12 MiB	9,985.35	14.65	9,970.70	14.65
15:41:34	25 MiB	9,960.94	39.06	9,921.88	39.06
15:50:04	50 MiB	9,912.11	87.89	9,824.22	87.89
15:53:29	100 MiB	9,814.45	185.55	9,628.90	185.55
16:45:03	200 MiB	9,619.14	380.86	9,238.28	380.86

The owner view reports both a debited balance and a reserved amount for each file, and the available figure is the balance less the reservation. The operator's payout reconciliation at 16:45 balanced exactly.

Table 16. Payment metrics (Run 3)

ID	Metric	Value	Remark
D-PAY-01	Ledger drift	0 paise across 7 checkpoints ^[D]	Integer-paise ledger
D-PAY-02	Charge cadence	<i>unmetered</i>	—
D-PAY-03	Reservation vs. monthly cost	equal at every checkpoint ^[D]	Rounded to the paisa per file
D-PAY-04	Payout reconciliation	exact ^[A]	Preview at 16:45
D-PAY-05	Hold → release latency	<i>unmetered</i>	—
D-PAY-06	Available reduction per ₹ of monthly cost	2.0 ^[D]	Balance debit plus reservation

5.8 Placement

Diversity cap. In every segment visible in the captured placement listings — 41 segments across the 3, 12, 50, 100 and 200 MiB files — the five fragments sat on five different synthetic ASNs, as the cap of one fragment per ASN per segment requires. No violation was observed.

Load distribution. Of the 2,620 file fragments stored by 16:47, providers held between 106 and 418 each (mean 291, coefficient of variation 0.41, Gini coefficient 0.22; Figure 11). Placement weighs reliability scores alongside the diversity cap. Two of the three least-loaded desks, 15F and 15E, received 18 of the first 41 repaired fragments after the departure in Run 4 (Section 5.6).

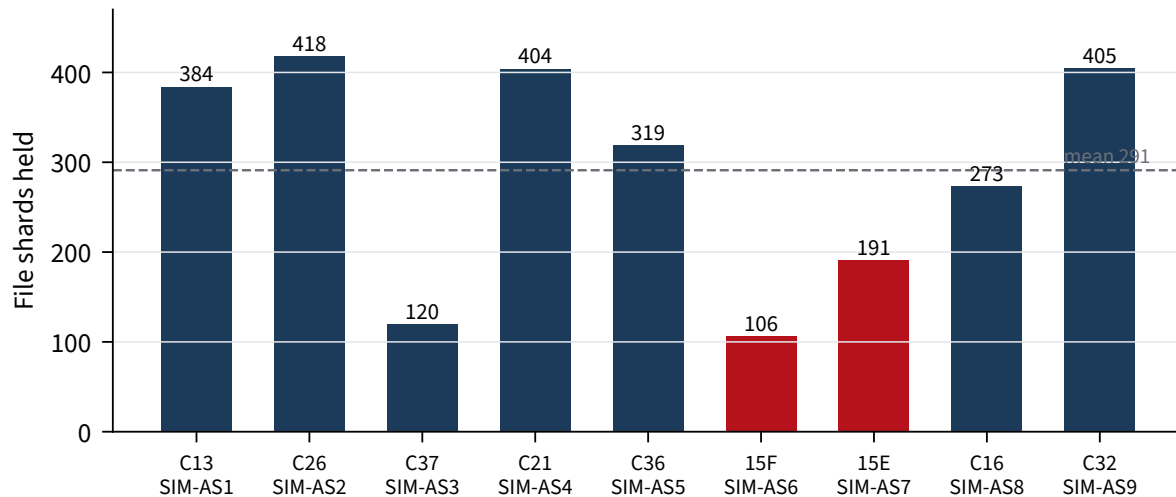


Figure 11. File fragments held per provider at 16:47 (vetting fragment excluded). Red: the two 4 GB desks.

Table 17. Placement metrics (Run 3)

ID	Metric	Value	Remark
D-PLC-01	Diversity-cap violations	0 in 41 segments inspected ^[M]	Synthetic ASNs
D-PLC-02	Fragments per provider	max 418, min 106, mean 291; CV 0.41; Gini 0.22 ^[A] / ^[D]	2,620 fragments, 9 providers
D-PLC-03	Active providers holding zero fragments	9 of 9 until 15:16; 0 after ^[A]	Expected before first upload
D-PLC-04	Existing fragments moved to a new provider	<i>unmetered</i>	Run 4 phase not reached

5.9 Churn

The churn phases of Run 4 — rejoining under a new identity, time to **ACTIVE** after rejoining, fragments received by a newly joined provider, and recovery onto a second device — were not reached. The only churn event measured is the announced departure of C32 (Section 5.6). Metrics D-CHN-01 to D-CHN-04 are unmetered.

5.10 Confidentiality and integrity

Operator surfaces were inspected throughout the run. The console and the operator's placement listing expose fragment identifiers (SHA-256 hashes), provider identifiers, synthetic ASNs and sizes, and no file content or file name (the display-name field reads *none set*); D-SEC-02 therefore holds by inspection ^[M]. The measurements of on-disk entropy of stored fragments (D-SEC-01), verified deletion (D-SEC-03) and refusal of uploads from a recovered device (D-SEC-04) were scheduled in Run 4 and are unmetered. Screenshots of account registration are not reproduced in this report.

5.11 Observability

All 21 dashboard panels displayed data during the run, including the three added for it; the repair-duration panel populated as soon as repair began. Measured from each target's first successful scrape to 16:50:00, scrape success was 99.85 % for the coordinator and 99.28–99.68 % for the providers. Heartbeats arrived at 1.95–2.05 per minute per desk against a configured two per minute. The assignment–delivery gap panel correctly went negative during vetting, because vetting fragments count as stored but not as assigned.

Table 18. Observability metrics (Run 3)

ID	Metric	Value	Remark
D-OBS-01	Dashboard panels live	21 of 21 ^[M]	Verified before the run
D-OBS-02	Scrape success	coordinator 99.85 %; providers 99.28–99.68 % ^[A]	From first successful scrape to 16:50
D-OBS-03	Heartbeat cadence	1.95–2.05 per minute per desk ^[A]	Configured 2 per minute
D-OBS-04	Client-side resource sampling	<i>unmetered</i>	—

6 Future directions and conclusion

6.1 Future directions

Quantities marked *unmetered* in this report are expected to be measured in subsequent revisions of it. Development of the system continues on a separate long-term-support (LTS) track, which will be described in its own documentation; this report makes no claims about that track.

6.2 Conclusion

On ten physically separate machines, the demonstration build of Vyomanaunt showed its core mechanisms working end to end. Providers onboarded and vetted themselves in about ten minutes at best; files from 3 to 200 MiB were encrypted, erasure-coded and placed under a network-diversity cap without the coordinator touching their bytes; every assigned fragment was confirmed stored; more than seventy thousand audits ran, with a pass rate of 98 % at rest and no integrity failures; the escrow ledger was exact to the paisa; and repair began automatically when a provider announced its departure. The coordinator and provider daemons did this with modest resources — 43 MB and at most 223 MB of memory respectively. This report is the reference record of the demonstration build as it stands.

Acknowledgements

The authors thank the staff of the campus computer laboratory that hosted the measurement rig, and everyone who gave their afternoon to operate the desks.

A Metric register

The register is frozen: identifiers are never renumbered, and new metrics are appended. Status: **R** reported in this version; **P** partially reported; **U** unmetered.

Table 19. Metric register and status in Version 1.0.0

ID	Metric	St.	ID	Metric	St.
D-ENV-01	Machine hardware	R	D-RES-09	Value-log append latency	R
D-ENV-02	Operating system and build	R	D-AUD-01	Challenges issued	R
D-ENV-03	Overlay version	R	D-AUD-02	Pass rate, steady state	R
D-ENV-04	Uplink type	R	D-AUD-03	Timeout rate	R
D-ENV-05	Network speed, two passes	P	D-AUD-04	Response latency p95	R
D-ENV-06	Source revision under test	U	D-AUD-05	Pass rate under load	R
D-ENV-07	Free disk at start	R	D-AUD-06	Content-hash failures	R
D-NET-01	Start → ready	R	D-REP-01	Departure detection latency	U
D-NET-02	Start → 100 % active	R	D-REP-02	Repair queue peak	R
D-NET-03	Join → active per provider	R	D-REP-03	Drain time	P
D-NET-04	One-time code latency	U	D-REP-04	Job duration p50/p95	R
D-NET-05	Overlay authorisation → address	U	D-REP-05	Completed / queued	P
D-NET-06	Join → target healthy	U	D-REP-06	Post-repair cap compliance	U
D-NET-07	Operator join cadence	R	D-REP-07	ASN-twin vs. non-twin repair	U
D-NET-08	Low-score active providers	R	D-DUR-01	Retrieval with providers dead	U
D-NET-09	Unaudited active providers	R	D-DUR-02	Fragments unrecoverable	U
D-UPL-01	Upload duration	R	D-DUR-03	Post-churn integrity	U
D-UPL-02	Upload throughput	R	D-PAY-01	Ledger drift	R
D-UPL-03	Segments and fragments	R	D-PAY-02	Charge cadence	U
D-UPL-04	Wire amplification	P	D-PAY-03	Reservation vs. cost	R
D-UPL-05	Stall duration	P	D-PAY-04	Payout reconciliation	P
D-UPL-06	Largest file	R	D-PAY-05	Hold → release	U
D-UPL-07	Assignment-delivery gap	R	D-PAY-06	Available per ₹ of cost	R
D-UPL-08	Resume behaviour	R	D-PLC-01	Diversity-cap violations	R
D-RET-01	Retrieval duration	R	D-PLC-02	Load spread	R
D-RET-02	Byte-exactness	U	D-PLC-03	Zero-fragment active providers	R
D-RET-03	Retrieval with providers dead	U	D-PLC-04	New-provider migration	U
D-RET-04	Cross-device recovery	U	D-CHN-01-04	Churn metrics	U
D-RET-05	Format fidelity	U	D-SEC-01	Stored-fragment entropy	U
D-RES-01	Client peak memory	P	D-SEC-02	No plaintext on operator surfaces	R
D-RES-02	Memory ÷ file size	P	D-SEC-03	Deletion verified	U
D-RES-03	Memory pressure / swap	R	D-SEC-04	Upload refused on recovered device	U
D-RES-04	CPU pattern	R	D-OBS-01	Panels live	R
D-RES-05	Coordinator memory / CPU	R	D-OBS-02	Scrape success	R
D-RES-06	Provider daemon memory	R	D-OBS-03	Heartbeat cadence	R
D-RES-07	Provider disk footprint	U	D-OBS-04	Client resource sampling	U
D-RES-08	RAM-constrained flags	R			

B Raw run data

Table 20. Scribe log, Run 3 (IST). [†]Corrected from the console event feed (Section 5.1); [‡]logged as “14:41”, seconds taken from the console event feed.

Desk	Join	Active	Event	Start	End
C13	14:13:45	14:26:56	Run start	14:00:32	—
C26	14:16:10	14:27:56	Coordinator start	14:06:02	—
C37	14:17:51	14:40:49	Ready	14:41:57	—
C21	14:19:37	14:31:20	100 % active	15:03:37	—
C36	14:23:26	14:55:35 [†]	3 MiB rung	15:16:02	15:28:18
15F	14:24:39	14:45:25	12 MiB rung	15:28:54	15:36:30
15E	14:29:28	14:41:18 [‡]	25 MiB rung	15:37:41	15:43:40
C16	14:30:35	15:03:37 [†]	50 MiB rung	15:47:41	15:50:44
C32	14:32:24	14:42:27	100 MiB rung	15:50:49	15:56:22
			200 MiB rung	15:56:53	16:40:56
			Providers lost (Prometheus)	16:50:15	—

Table 21. Assignment counter and confirmed stores at rung boundaries (Prometheus, 15 s samples) ^[A]

Sample (IST)	Assigned (cum.)	Stored, excl. vetting (cum.)	Gap
15:10:00	0	0	0
15:17:00 (after 3 MiB)	25	25	0
15:30:15 (after 12 MiB)	110	110	0
15:38:15 (after 25 MiB)	280	280	0
15:47:30 (after 50 MiB)	615	615	0
15:53:00 (after 100 MiB)	1,285	1,285	0
15:57:15 (200 MiB assigned)	2,620	1,285	1,335
16:25:00 (first attempt ends)	2,620	2,595	25
16:40:00 (after second resumption)	2,620	2,620	0
16:50:00 (repair in progress)	2,620	2,661	−41

Glossary

AES-256-CTR The Advanced Encryption Standard with a 256-bit key in counter mode, which turns the block cipher into a stream cipher. One of the two ciphers used inside the all-or-nothing transform.

AES-NI Processor instructions that accelerate AES. The client selects AES-256-CTR when they are present and ChaCha20 otherwise.

All-or-nothing transform (AONT) A keyless, invertible transform whose output cannot be inverted unless essentially all of it is available. Applied before erasure coding in the AONT-RS construction [9], so that fewer than k fragments reveal nothing practical about a segment and no key has to be stored.

Argon2id A memory-hard password-hashing function. Derives the owner's master secret from the account passphrase.

ASN (autonomous system number) An identifier for an independently operated network, such as an Internet service provider. Used by placement to spread each segment across networks; synthetic in the measured run (SIM-AS1...SIM-AS9).

ASN cap The maximum fraction of any one segment's fragments that may be placed on a single ASN; 0.2 in the demonstration profile, i.e. one fragment per ASN per segment.

Assignment-delivery gap Fragments the coordinator has assigned minus fragments providers have confirmed storing, excluding vetting fragments. A measure of delivery latency.

Audit challenge A request from the coordinator to a provider to prove possession of a specific stored fragment, bound to a fresh nonce and a deadline. Outcomes are **PASS**, **FAIL** and **TIMEOUT**.

BadgerDB An embedded key-value store written in Go that separates keys from values in the manner of WiscKey. The provider storage engine on Windows.

BIP-39 mnemonic A sequence of words from a standard list that encodes a secret in human-readable form. The client shows a 24-word mnemonic at registration for account recovery.

Canary A fixed value appended to each segment before the all-or-nothing transform and checked after decoding, so that a corrupted segment is detected rather than returned.

ChaCha20-Poly1305 An authenticated-encryption construction combining the ChaCha20 stream cipher with the Poly1305 authenticator (AEAD_CHACHA20_POLY1305). Seals pointer files; ChaCha20 alone is the alternative cipher inside the transform.

Chunk A fragment at rest on a provider, addressed by the SHA-256 hash of its content.

Coordinator The coordinating service of Vyomanaunt, backed by PostgreSQL. It admits and vets providers, places fragments, audits, schedules repair and keeps the escrow ledger, but never carries file data.

CRDT (conflict-free replicated data type) A data structure whose replicas can be updated independently and merged without conflict. The escrow ledger follows the positive-negative counter pattern: balances are derived from sums of append-only events.

Data owner A user who stores files on the network through the client.

Deduplication Storing identical content only once. Vyomanaunt does not deduplicate: every upload uses fresh randomness, so identical files produce unrelated fragments.

Demonstration profile The set of parameter values (Table 2) that compresses the system's time constants so that its whole lifecycle can be observed in an afternoon.

Departure A provider leaving the network, either by announcing it or by remaining silent beyond the departure threshold. Its fragments become repair jobs.

DHT (distributed hash table) A key-value lookup service spread across peers. Vyomanaunt uses a Kademlia-style DHT for fragment-address lookup.

Ed25519 An elliptic-curve digital-signature scheme. Identifies owners and providers, authenticates transport connections and signs audit receipts.

Encryption Transforming data so that only holders of the appropriate key can read it. In Vyomanaunt all encryption of file data happens on the owner's device.

Erasur coding (Reed–Solomon) Encoding data into n fragments of which any k suffice to reconstruct it. The demonstration profile uses a Reed–Solomon code with $k = 3$ data and 2 parity fragments [7, 32].

Escrow and escrow ledger Owner funds held by the system and released to providers in proportion to audits passed, recorded as append-only events in integer paise.

Fragment (shard) One of the n outputs of erasure-coding a segment; 256 KiB in every profile.

Goroutine A lightweight thread of execution managed by the Go runtime. Provider daemons ran 31–32 each.

Grafana An open-source dashboarding tool; displays the Prometheus metrics of the run in 21 panels.

Heartbeat A periodic liveness message from a provider daemon to the coordinator; every 30 s with 5 s jitter in the demonstration profile.

HKDF-SHA256 The HMAC-based key-derivation function instantiated with SHA-256. Derives purpose-specific keys from the owner's master secret.

Kademlia DHT A distributed hash table in which peers and keys share an identifier space and distance is the bitwise XOR of identifiers, giving logarithmic lookup [19, 70].

libp2p A modular peer-to-peer networking stack. The demonstration build derives peer identifiers in the libp2p format but does not use the library.

MiB Mebibyte, 2^{20} bytes; distinct from the megabyte (MB, 10^6 bytes).

Mock payment mode A payment mode in which ledger entries are recorded exactly as in live operation but no money moves.

NAT and NAT traversal Network address translation lets many devices share one public address but blocks unsolicited inbound connections; NAT traversal is the set of techniques (probing, hole punching, relaying) that establish peer-to-peer connections across it. In the measured run ZeroTier performed traversal.

Nash equilibrium In game theory, a combination of strategies in which no participant can gain by changing its own strategy alone. Used in analysing whether honest auditing and storage are stable behaviours [62, 63].

NetworkProfile The single configuration structure that holds every parameter differing between profiles, printed by each program at start-up.

Operator console The terminal user interface through which an operator watches the readiness gate, fleet, repair, audit, escrow and events (Figure 5).

Percentile latency (p50, p95) The value below which the stated percentage of observed latencies fall: p50 is the median and p95 the 95th percentile.

Paise One hundredth of a rupee; the ledger's unit of account.

Pointer file Per-file metadata — which providers hold which fragments, and the encoding parameters — sealed on the owner's device and stored at the coordinator as ciphertext it cannot open.

Prometheus An open-source monitoring system that scrapes numeric metrics from programs at fixed intervals and stores them as time series; every 15 s in the run.

Provider A desktop computer running the Vyomanaunt daemon and contributing storage in return for payment.

Razorpay An Indian payment gateway. The demonstration build contains a Razorpay adapter; the readiness gate counts provider payout accounts past their cooling period.

Readiness gate The set of fleet conditions — active vetted providers, distinct ASNs, metropolitan regions, relay nodes and cooled payout accounts — that must hold before uploads are accepted.

Relay nodes (circuit relay) Publicly reachable peers that forward traffic between two peers that cannot connect directly. The demonstration profile requires none.

Reliability score A provider's score computed from its audit outcomes over short, medium and long sliding windows; an input to placement.

Repair Reconstructing a lost fragment from any k surviving fragments of its segment and storing it on a new provider under the same placement rules.

RocksDB An embedded key-value store based on a log-structured merge tree [72]; the index of the WiscKey engine on macOS and Linux.

Scrape One collection of a program's metrics by Prometheus.

Segment The unit of a file that the client encodes independently; 786,384 bytes of plaintext in the demonstration profile.

Shard size The size of one fragment: 262,144 bytes (256 KiB), fixed across profiles.

TLS 1.3 Version 1.3 of the Transport Layer Security protocol; secures every fragment transfer in the demonstration build, with certificates bound to Ed25519 identities.

TRAI MySpeed The Telecom Regulatory Authority of India's network speed-test application, used to characterise the rig's links.

Unmetered parameter A quantity this report defines but which the measured runs did not capture; printed as *unmetered* and never estimated.

Vetting and vetting period The probation a new provider serves, holding only a synthetic fragment and passing a minimum number of audits over a minimum time before becoming **ACTIVE**.

vLog (value log) The append-only file in which a WiscKey-style store keeps large values, with only small keys and pointers in its index; appended by a single writer.

WiscKey A key-value store design that separates keys from values to reduce write amplification for large values [73].

Working set The Windows measure of a process's resident memory, which the operating system can trim.

ZeroTier A software-defined virtual network that joins machines into one private overlay, handling NAT traversal and encryption between them. All ten machines in the run were joined by one ZeroTier network.

References

- [1] W. J. Bolosky, J. R. Douceur, D. Ely, and M. Theimer, “Feasibility of a serverless distributed file system deployed on an existing set of desktop PCs,” *SIGMETRICS Perform. Eval. Rev.*, vol. 28, no. 1, pp. 34–43, Jun. 2000, doi: [10.1145/345063.339345](https://doi.org/10.1145/345063.339345).
- [2] M. J. Litzkow, M. Livny, and M. W. Mutka, “Condor – A hunter of idle workstations,” in *Proc. 8th Int. Conf. Distributed Computing Systems*, Jun. 1988, pp. 104–111, doi: [10.1109/DCS.1988.12507](https://doi.org/10.1109/DCS.1988.12507).
- [3] D. P. Anderson, “BOINC: A system for public-resource computing and storage,” in *Proc. 5th IEEE/ACM Int. Workshop on Grid Computing*, Nov. 2004, pp. 4–10, doi: [10.1109/GRID.2004.14](https://doi.org/10.1109/GRID.2004.14).
- [4] R. Bhagwan, S. Savage, and G. M. Voelker, “Understanding availability,” in *Peer-to-Peer Systems II (IPTPS 2003)*, M. F. Kaashoek and I. Stoica, Eds. Berlin, Heidelberg: Springer, 2003, pp. 256–267, doi: [10.1007/978-3-540-45172-3_24](https://doi.org/10.1007/978-3-540-45172-3_24).
- [5] S. Saroiu, P. K. Gummadi, and S. D. Gribble, “Measurement study of peer-to-peer file sharing systems,” in *Multimedia Computing and Networking 2002*, SPIE, Dec. 2001, p. 156, doi: [10.1117/12.449977](https://doi.org/10.1117/12.449977).
- [6] C. Blake and R. Rodrigues, “High availability, scalable storage, dynamic peer networks: Pick two,” in *Proc. 9th Workshop on Hot Topics in Operating Systems (HotOS IX)*, Lihue, HI, USA: USENIX, May 2003. [Online]. Available: https://www.usenix.org/legacy/events/hotos03/tech/full_papers/blake/blake_html/
- [7] A. G. Dimakis, P. B. Godfrey, Y. Wu, M. J. Wainwright, and K. Ramchandran, “Network coding for distributed storage systems,” *IEEE Trans. Inf. Theory*, vol. 56, no. 9, pp. 4539–4551, Sep. 2010, doi: [10.1109/TIT.2010.2054295](https://doi.org/10.1109/TIT.2010.2054295).
- [8] F. Giroire, J. Monteiro, and S. Pérennes, “Peer-to-peer storage systems: A practical guideline to be lazy,” in *Proc. IEEE Global Telecommunications Conf. (GLOBECOM)*, Dec. 2010, pp. 1–6, doi: [10.1109/GLOCOM.2010.5683761](https://doi.org/10.1109/GLOCOM.2010.5683761).
- [9] J. K. Resch and J. S. Plank, “AONT-RS: Blending security and performance in dispersed storage systems,” in *Proc. 9th USENIX Conf. File and Storage Technologies (FAST ’11)*, San Jose, CA, USA, Feb. 2011, p. 14.
- [10] Z. Wilcox-O’Hearn and B. Warner, “Tahoe: The least-authority filesystem,” in *Proc. 4th ACM Int. Workshop on Storage Security and Survivability (StorageSS ’08)*. New York, NY, USA: ACM, Oct. 2008, pp. 21–26, doi: [10.1145/1456469.1456474](https://doi.org/10.1145/1456469.1456474).
- [11] G. Ateniese *et al.*, “Provable data possession at untrusted stores,” in *Proc. 14th ACM Conf. Computer and Communications Security (CCS ’07)*. New York, NY, USA: ACM, Oct. 2007, pp. 598–609, doi: [10.1145/1315245.1315318](https://doi.org/10.1145/1315245.1315318).
- [12] A. Juels and B. S. Kaliski, “PORs: Proofs of retrievability for large files,” in *Proc. 14th ACM Conf. Computer and Communications Security (CCS ’07)*. New York, NY, USA: ACM, Oct. 2007, pp. 584–597, doi: [10.1145/1315245.1315317](https://doi.org/10.1145/1315245.1315317).
- [13] B. Cohen, “Incentives build robustness in BitTorrent,” in *Proc. 1st Workshop on Economics of Peer-to-Peer Systems*, Berkeley, CA, USA, May 2003.
- [14] C. Ihle, D. Trautwein, M. Schubotz, N. Meuschke, and B. Gipp, “Incentive mechanisms in peer-to-peer networks – A systematic literature review,” *ACM Comput. Surv.*, vol. 56, no. 1, Jan. 2023. [Online]. Available: <https://dl.acm.org/doi/full/10.1145/3578581>
- [15] Storj Labs, Inc., “Storj: A decentralized cloud storage network framework,” White paper, v3.0, Oct. 2018.
- [16] Protocol Labs, “Filecoin: A decentralized storage network,” White paper, Jul. 2017.
- [17] J. Benet, “IPFS – Content addressed, versioned, P2P file system,” arXiv:1407.3561, Jul. 2014, doi: [10.48550/arXiv.1407.3561](https://doi.org/10.48550/arXiv.1407.3561).

- [18] C. Li, M. Xu, J. Zhang, H. Guo, and X. Cheng, “SoK: Decentralized storage network,” *High-Confidence Computing*, vol. 4, no. 3, Art. 100239, Sep. 2024, doi: 10.1016/j.hcc.2024.100239.
- [19] P. Maymounkov and D. Mazières, “Kademlia: A peer-to-peer information system based on the XOR metric,” in *Peer-to-Peer Systems (IPTPS 2002)*, P. Druschel, F. Kaashoek, and A. Rowstron, Eds. Berlin, Heidelberg: Springer, 2002, pp. 53–65, doi: 10.1007/3-540-45748-8_5.
- [20] D. Trautwein *et al.*, “Design and evaluation of IPFS: A storage layer for the decentralized web,” in *Proc. ACM SIGCOMM 2022 Conf.* New York, NY, USA: ACM, Aug. 2022, pp. 739–752, doi: 10.1145/3544216.3544232.
- [21] S. Rhea, D. Geels, T. Roscoe, and J. Kubiatowicz, “Handling churn in a DHT,” in *Proc. USENIX Annual Technical Conf. (ATC)*, Boston, MA, USA, Jun. 2004. [Online]. Available: https://www.usenix.org/legacy/event/usenix04/tech/general/rhea/rhea_html/usenix-cr.html
- [22] S. Nath, H. Yu, P. B. Gibbons, and S. Seshan, “Subtleties in tolerating correlated failures in wide-area storage systems,” in *Proc. 3rd USENIX Symp. Networked Systems Design and Implementation (NSDI ’06)*, 2006. [Online]. Available: https://www.usenix.org/legacy/event/nsdi06/tech/full_papers/nath/nath_html/
- [23] O. Dalle, F. Giroire, J. Monteiro, and S. Pérennes, “Analysis of failure correlation in peer-to-peer storage systems,” INRIA, Research Report RR-6771, 2008. [Online]. Available: <https://inria.hal.science/inria-00346857>
- [24] V. Prabhakaran, L. N. Bairavasundaram, N. Agrawal, H. S. Gunawi, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “IRON file systems,” in *Proc. 20th ACM Symp. Operating Systems Principles (SOSP ’05)*. New York, NY, USA: ACM, Oct. 2005, pp. 206–220, doi: 10.1145/1095810.1095830.
- [25] B. Schroeder, R. Lagisetty, and A. Merchant, “Flash reliability in production: The expected and the unexpected,” in *Proc. 14th USENIX Conf. File and Storage Technologies (FAST ’16)*, 2016, pp. 67–80. [Online]. Available: <https://www.usenix.org/conference/fast16/technical-sessions/presentation/schroeder>
- [26] S. Goparaju, A. Fazeli, and A. Vardy, “Minimum storage regenerating codes for all parameters,” *IEEE Trans. Inf. Theory*, vol. 63, no. 10, pp. 6318–6328, Oct. 2017, doi: 10.1109/TIT.2017.2690662.
- [27] V. Guruswami and M. Wootters, “Repairing Reed–Solomon codes,” in *Proc. 48th Annu. ACM Symp. Theory of Computing (STOC)*, Cambridge, MA, USA: ACM, Jun. 2016, pp. 216–226, doi: 10.1145/2897518.2897525.
- [28] K. V. Rashmi, N. B. Shah, D. Gu, H. Kuang, D. Borthakur, and K. Ramchandran, “A ‘Hitchhiker’s’ guide to fast and efficient data reconstruction in erasure-coded data centers,” in *Proc. ACM SIGCOMM 2014 Conf.*, Chicago, IL, USA, 2014. [Online]. Available: <https://dl.acm.org/doi/abs/10.1145/2619239.2626325>
- [29] K. Cheng, G. Li, X. Li, S. Hu, and P. P. C. Lee, “LESS is more for I/O-efficient repairs in erasure-coded storage,” in *Proc. 24th USENIX Conf. File and Storage Technologies (FAST ’26)*, 2026, pp. 631–641. [Online]. Available: <https://www.usenix.org/conference/fast26/presentation/cheng>
- [30] M. Elyasi and S. Mohajer, “Cascade codes for distributed storage systems,” *IEEE Trans. Inf. Theory*, vol. 66, no. 12, pp. 7490–7527, Dec. 2020, doi: 10.1109/TIT.2020.3033338.
- [31] M. Silberstein, L. Ganesh, Y. Wang, L. Alvisi, and M. Dahlin, “Lazy means smart: Reducing repair bandwidth costs in erasure-coded distributed storage,” in *Proc. Int. Conf. Systems and Storage (SYSTOR ’14)*, 2014. [Online]. Available: <https://dl.acm.org/doi/abs/10.1145/2611354.2611370>
- [32] Z. Shen, Y. Cai, K. Cheng, P. P. C. Lee, X. Li, Y. Hu, and J. Shu, “A survey of the past, present, and future of erasure coding for storage systems,” *ACM Trans. Storage*, vol. 21, no. 1, Art. 4, Jan. 2025. [Online]. Available: <https://dl.acm.org/doi/full/10.1145/3708994>

- [33] C. Yin, Z. Xu, W. Li, T. Li, S. Yuan, and Y. Liu, “Erasure codes for cold data in distributed storage systems,” *Applied Sciences*, vol. 13, no. 4, Art. 2170, Feb. 2023. [Online]. Available: <https://www.mdpi.com/2076-3417/13/4/2170>
- [34] J. Li and B. Li, “Zebra: Demand-aware erasure coding for distributed storage systems,” in *Proc. IEEE/ACM 24th Int. Symp. Quality of Service (IWQoS)*, Jun. 2016, pp. 1–10, doi: 10.1109/IWQoS.2016.7590388.
- [35] J. Chen, N. Wang, Z. Li, Z. Liu, and D. Xiang, “HALO: A scalable framework for hotness-aware coding and transformation-efficient placement,” *Future Generation Computer Systems*, vol. 183, Art. 108562, Oct. 2026, doi: 10.1016/j.future.2026.108562.
- [36] M. Zhang, Q. Wang, Z. Shen, and P. P. C. Lee, “POCache: Toward robust and configurable straggler tolerance with parity-only caching,” *J. Parallel Distrib. Comput.*, vol. 167, pp. 157–172, Sep. 2022, doi: 10.1016/j.jpdc.2022.05.004.
- [37] Y. Ren, Y. Ren, X. Li, Y. Hu, J. Li, and P. P. C. Lee, “ELECT: Enabling erasure coding tiering for LSM-tree-based storage,” in *Proc. 22nd USENIX Conf. File and Storage Technologies (FAST ’24)*, 2024, pp. 293–310. [Online]. Available: <https://www.usenix.org/conference/fast24/presentation/ren>
- [38] J. Chen, Z. Hu, C. Wang, Q. Zhou, and X. Zhang, “A bi-level programming model to enhance the encoding efficiency of erasure codes,” *Expert Systems with Applications*, vol. 331, Art. 133334, Dec. 2026, doi: 10.1016/j.eswa.2026.133334.
- [39] R. Friedman, R. Kapelko, and K. Marchwicki, “On the data persistency of replicated erasure codes in distributed storage systems,” *Information and Computation*, vol. 304, Art. 105297, May 2025, doi: 10.1016/j.ic.2025.105297.
- [40] H. Krawczyk, “Secret sharing made short,” in *Advances in Cryptology – CRYPTO ’93*, LNCS vol. 773. Berlin, Heidelberg: Springer, 1994, pp. 136–146. [Online]. Available: https://link.springer.com/chapter/10.1007/3-540-48329-2_12
- [41] M. Szabó, Á. Recse, R. Szabó, D. Balla, and M. Maliosz, “Separation and optimization of encryption and erasure coding in decentralized storage systems,” *Future Generation Computer Systems*, vol. 167, Art. 107739, Jun. 2025, doi: 10.1016/j.future.2025.107739.
- [42] S. Goparaju, S. E. Rouayheb, R. Calderbank, and H. V. Poor, “Data secrecy in distributed storage systems under exact repair,” in *Proc. Int. Symp. Network Coding (NetCod)*, Jun. 2013, pp. 1–6, doi: 10.1109/NetCod.2013.6570831.
- [43] R. Tandon, S. Amuru, T. C. Clancy, and R. M. Buehrer, “Toward optimal secure distributed storage systems with exact repair,” *IEEE Trans. Inf. Theory*, vol. 62, no. 6, pp. 3477–3492, Jun. 2016, doi: 10.1109/TIT.2016.2544340.
- [44] S. Shao, T. Liu, C. Tian, and C. Shen, “On the tradeoff region of secure exact-repair regenerating codes,” *IEEE Trans. Inf. Theory*, vol. 63, no. 11, pp. 7253–7266, Nov. 2017, doi: 10.1109/TIT.2017.2750156.
- [45] A. S. Rawat, “Secrecy capacity of minimum storage regenerating codes,” in *Proc. IEEE Int. Symp. Information Theory (ISIT)*, Aachen, Germany, Jun. 2017, pp. 1406–1410, doi: 10.1109/ISIT.2017.8006760.
- [46] V. A. Rameshwar and N. Kashyap, “Achieving secrecy capacity of minimum storage regenerating codes for all feasible (n, k, d) parameter values,” in *Proc. National Conf. Communications (NCC)*, Feb. 2019, pp. 1–6, doi: 10.1109/NCC.2019.8732243.
- [47] M. Kleckler and S. Mohajer, “Secure determinant codes: A class of secure exact-repair regenerating codes,” in *Proc. IEEE Int. Symp. Information Theory (ISIT)*, Paris, France, Jul. 2019, pp. 211–215, doi: 10.1109/ISIT.2019.8849391.

- [48] K. Huang, U. Parampalli, and M. Xian, "Security concerns in minimum storage cooperative regenerating codes," *IEEE Trans. Inf. Theory*, vol. 62, no. 11, pp. 6218–6232, Nov. 2016, doi: 10.1109/TIT.2016.2608949.
- [49] K. V. Rashmi, N. B. Shah, K. Ramchandran, and P. V. Kumar, "Information-theoretically secure erasure codes for distributed storage," *IEEE Trans. Inf. Theory*, vol. 64, no. 3, pp. 1621–1646, Mar. 2018, doi: 10.1109/TIT.2017.2769101.
- [50] A. S. Rawat, O. O. Koyluoglu, N. Silberstein, and S. Vishwanath, "Optimal locally repairable and secure codes for distributed storage systems," *IEEE Trans. Inf. Theory*, vol. 60, no. 1, pp. 212–236, Jan. 2014, doi: 10.1109/TIT.2013.2288784.
- [51] W. Huang and J. Bruck, "Generic secure repair for distributed storage," arXiv:1706.00500, Jun. 2017, doi: 10.48550/arXiv.1706.00500.
- [52] H. Shacham and B. Waters, "Compact proofs of retrievability," *J. Cryptology*, Springer, 2013. [Online]. Available: <https://link.springer.com/article/10.1007/s00145-012-9129-2>
- [53] C. C. Erway, A. Küpçü, C. Papamanthou, and R. Tamassia, "Dynamic provable data possession," *ACM Trans. Inf. Syst. Secur.*, vol. 17, no. 4, Art. 15, Apr. 2015. [Online]. Available: <https://dl.acm.org/doi/abs/10.1145/2699909>
- [54] F. Armknecht, J.-M. Bohli, G. O. Karame, Z. Liu, and C. A. Reuter, "Outsourced proofs of retrievability," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS '14)*. New York, NY, USA: ACM, Nov. 2014, pp. 831–843, doi: 10.1145/2660267.2660310.
- [55] T. Li, J. Chu, and L. Hu, "CIA: A collaborative integrity auditing scheme for cloud data with multi-replica on multi-cloud storage providers," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 1, pp. 154–162, Jan. 2023, doi: 10.1109/TPDS.2022.3216614.
- [56] H. Zeng, L. Chen, Z. Gu, S. Wu, J. Pan, and Y. Yan, "Post-quantum auditing for outsourced storage with probabilistic guarantees," *Peer-to-Peer Netw. Appl.*, vol. 19, no. 4, Art. 93, May 2026, doi: 10.1007/s12083-026-02243-5.
- [57] B. Chen, R. Curtmola, G. Ateniese, and R. Burns, "Remote data checking for network coding-based distributed storage systems," in *Proc. ACM Workshop on Cloud Computing Security (CCSW '10)*. New York, NY, USA: ACM, Oct. 2010, pp. 31–42, doi: 10.1145/1866835.1866842.
- [58] B. Chen, A. K. Ammala, and R. Curtmola, "Towards server-side repair for erasure coding-based distributed storage systems," in *Proc. 5th ACM Conf. Data and Application Security and Privacy (CODASPY '15)*, 2015, pp. 281–288. [Online]. Available: <https://dl.acm.org/doi/abs/10.1145/2699026.2699122>
- [59] B. Chen and R. Curtmola, "Remote data integrity checking with server-side repair," *J. Computer Security*, vol. 25, no. 6, pp. 537–584, 2017. [Online]. Available: <https://journals.sagepub.com/doi/abs/10.3233/JCS-16868>
- [60] S. D. Kamvar, M. T. Schlosser, and H. Garcia-Molina, "The EigenTrust algorithm for reputation management in P2P networks," in *Proc. 12th Int. Conf. World Wide Web (WWW '03)*, Budapest, Hungary, 2003. [Online]. Available: <https://dl.acm.org/doi/abs/10.1145/775152.775242>
- [61] L. Xiong and L. Liu, "PeerTrust: Supporting reputation-based trust for peer-to-peer electronic communities," *IEEE Trans. Knowl. Data Eng.*, vol. 16, no. 7, pp. 843–857, Jul. 2004, doi: 10.1109/TKDE.2004.1318566.
- [62] C. Buragohain, D. Agrawal, and S. Suri, "A game theoretic framework for incentives in P2P systems," in *Proc. 3rd Int. Conf. Peer-to-Peer Computing (P2P 2003)*, Sep. 2003, pp. 48–56, doi: 10.1109/PTP.2003.1231503.
- [63] I. Vakulinia, W. Wang, and J. Xin, "An incentive-compatible mechanism for decentralized storage network," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 4, pp. 2294–2306, Jul. 2023, doi: 10.1109/TNSE.2023.3245326.

- [64] M. Crystal, G. Goren, and S. D. Kominers, “Rationally analyzing Shelby: Proving incentive compatibility in a decentralized storage network,” arXiv:2510.11866, Oct. 2025, doi: 10.48550/arXiv.2510.11866.
- [65] E. P. Lazear, “Why is there mandatory retirement?,” *J. Political Economy*, vol. 87, no. 6, pp. 1261–1284, Dec. 1979, doi: 10.1086/260835.
- [66] A. Rosenblat and L. Stark, “Algorithmic labor and information asymmetries: A case study of Uber’s drivers,” *Int. J. Commun.*, vol. 10, pp. 3758–3784, Jul. 2016.
- [67] A. Voskoboynikov, O. Wiese, M. Mehrabi Koushki, V. Roth, and K. Beznosov, “The U in crypto stands for usable: An empirical study of user experience with mobile cryptocurrency wallets,” in *Proc. CHI Conf. Human Factors in Computing Systems (CHI ’21)*, 2021, doi: 10.1145/3411764.3445407.
- [68] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, and W. Vogels, “Dynamo: Amazon’s highly available key-value store,” *ACM SIGOPS Oper. Syst. Rev.*, vol. 41, no. 6, pp. 205–220, Oct. 2007 (Proc. SOSP ’07). [Online]. Available: <https://dl.acm.org/doi/abs/10.1145/1323293.1294281>
- [69] P. Bailis, A. Fekete, M. J. Franklin, A. Ghodsi, J. M. Hellerstein, and I. Stoica, “Coordination avoidance in database systems (extended version),” arXiv:1402.2237, Oct. 2014, doi: 10.48550/arXiv.1402.2237.
- [70] I. Baumgart and S. Mies, “S/Kademlia: A practicable approach towards secure key-based routing,” in *Proc. Int. Conf. Parallel and Distributed Systems (ICPADS)*, Dec. 2007, pp. 1–8, doi: 10.1109/ICPADS.2007.4447808.
- [71] D. Trautwein, C. Ihle, M. Schubotz, and B. Gipp, “Challenging tribal knowledge – Large scale measurement campaign on decentralized NAT traversal,” arXiv:2510.27500, Oct. 2025, doi: 10.48550/arXiv.2510.27500.
- [72] P. O’Neil, E. Cheng, D. Gawlick, and E. O’Neil, “The log-structured merge-tree (LSM-tree),” *Acta Informatica*, vol. 33, no. 4, pp. 351–385, Jun. 1996, doi: 10.1007/s002360050048.
- [73] L. Lu, T. S. Pillai, H. Gopalakrishnan, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “WiscKey: Separating keys from values in SSD-conscious storage,” *ACM Trans. Storage*, vol. 13, no. 1, pp. 5:1–5:28, Mar. 2017, doi: 10.1145/3033273.